

Spectral clustering based mutant reduction for mutation testing

Changqing Wei^a, Xiangjuan Yao^{a,*}, Dunwei Gong^b, Huai Liu^c

^a School of Mathematics, China University of Mining and Technology, Xuzhou 221116, China

^b School of Information and Control Engineering, China University of Mining and Technology, Xuzhou 221116, China

^c Department of Computer Science and Software Engineering, Swinburne University of Technology, Melbourne, Australia

ARTICLE INFO

Keywords:

Mutation testing
Spectral clustering
Mutant reduction

ABSTRACT

Context: Mutation testing techniques, which attempt to construct a set of so-called mutants by seeding various faults into the software under test, have been widely used to generate test cases as well as to evaluate the effectiveness of a test suite. Its popularity in practice is significantly hindered by its high cost, majorly caused by the large number of mutants generated by the technique.

Objective: It is always a challenging task to reduce the number of mutants while preserving the effectiveness of mutation testing. In this paper, we make use of an intelligent technique, namely spectral clustering, to improve the efficacy of mutant reduction.

Method: First of all, we give a family of definitions and the method to calculate the distance between mutants according to the weak mutation testing criteria. Then we propose a mutant reduction method based on spectral clustering (SCMT), including the determination method of the number of clusters, spectral clustering of mutants, and selection of representative mutants.

Results: The experimental studies based on 12 object programs show that the new approach can significantly reduce the number of mutants without jeopardizing the performance of mutation testing. As compared with other benchmark techniques, the new approach based on weak mutation testing criteria cannot only consistently deliver high effectiveness of mutation testing, but also help significantly reduce the time-cost of mutation testing.

Conclusion: It is clearly demonstrated that the use of spectral clustering can help enhance the cost-effectiveness of mutation testing. The research reveals some potential research directions for not only mutation testing but also the broad area of software testing.

1. Introduction

Software testing, an essential activity to ensure the quality of software, has received widespread attention in the community of software reliability. Nowadays, the scale and complexity of software are rapidly increased; as a consequence, software testing always incurs a high cost, e.g., a lot of time and resources. It is thus a challenging job to effectively improve the quality of software while reducing the testing cost [1].

Mutation testing [2,3] is a fundamental software testing technique. It applies a set of so-called mutation operators into different statements of the base program. As a result, various faults are seeded, and a family of faulty programs, termed as mutants, are generated. Mutation testing can simulate various possible practical faults of the program, measure the fault detection ability of a given test suite, and evaluate the adequacy of the test suite [4]. Due to its inception, different mutation testing techniques have been developed and applied into various domains [5,6]. Test data generated by mutation testing have shown a

high fault detection capability [7–9]. A number of studies [10,11] have also demonstrated that compared with manually fault-seeded programs, automatically generated mutants are more similar to the real-life faulty programs. Thus, mutation testing has been acknowledged as an effective technique for evaluating the fault-detection capability of a testing method.

However, the real-world application of mutation testing is hindered by some drawbacks, such as the existence of equivalent mutants [12], lack of appropriate automated tools, etc. One major drawback is the high computational cost incurred by large number of the mutants in mutation testing. A more description is as follows: There normally exist a large number of mutation operators, which can be applied into numerous locations in the base program. As a consequence, a huge amount of mutants are often produced by mutation testing, even for a small-sized program. It is normally quite time-consuming to execute mutation testing.

* Corresponding author.

E-mail address: yaqxj@cumt.edu.cn (X. Yao).

In order to reduce the computational cost incurred by the large number of mutants in mutation testing, researchers have proposed some approaches for reducing mutants, including those based on random sampling [13,14], operator selection [15,16] and clustering [12]. Among all these methods, the clustering based one [12], as the name suggests, makes use of clustering techniques (a major branch of machine learning) to classify mutants based on their similarities. Representative mutants are then selected from these clusters to compose a subset of mutants. A number of studies [12,17–19] have demonstrated that the clustering based mutant reduction would not affect the effectiveness of mutation testing very much — the same test suite would achieve similar mutant killing capabilities that was normally measured by the so-called mutation score on both the original set and the reduced subset of mutants. However, the existing methods have their own problems. For example, in Hussain’s method [12], the use of *K*-means algorithm or aggregate clustering algorithm makes it difficult to determine the number of clusters and the initial position of the cluster center. Such difficulty, in turn, affects the scalability of clustering based methods in mutant reduction, especially for large-scale programs. In addition, although GA [17] that is a mutant reduction method based on genetic algorithm can automatically determine the number of clusters, but the use of genetic algorithm must inevitably need to set some parameters, which will lead to a higher dependence of the results on the parameters.

In view of this, this paper makes use of an intelligent technique, namely spectral clustering, to reduce the number of mutants and solve some problems of existing clustering techniques, e.g., manually determining the number of clusters, the number of clusters is not accurate and the quality of clustering is limited. First, we give the definition of the distance of mutants and propose a calculation method. Then, we investigate the application of spectral clustering into mutant reduction. Different from previous clustering techniques, which normally directly cluster samples, spectral clustering makes use of eigenvalues and eigenvectors to indirectly cluster samples. In this way, we can reduce the calculation cost in the process of mutation testing by dimension reduction. In addition, it can achieve clustering in sample space of arbitrary shape and convergence to the global optimal solution. Furthermore, the new mutant reduction technique based on the spectral clustering can automatically determine the number of clusters, and thus improve the scalability across different scales of programs.

Finally, to evaluate the method presented in this paper, we select 12 Java programs as the subjects and apply two commonly used mutation testing tools, namely Muclipse [20] and MuJava’s plug-in on Eclipse [21], to generate mutants for all 12 Java programs. Our experiments are designed to answer three research questions. The experimental results show that the new approach can significantly reduce the number of mutants without jeopardizing the performance of mutation testing. As compared with other benchmark techniques, the new approach can consistently deliver high effectiveness of mutation testing. It is clearly demonstrated that the use of spectral clustering can help enhance the cost-effectiveness of mutation testing. The research reveals some potential research directions for not only mutation testing but also the broad area of software testing.

In particular, our work makes the following three contributions:

- (1) We give the definition of the distance of mutants and propose a method to calculate it;
- (2) We apply an intelligent technique, namely spectral clustering to mutant reduction for the first time and improve it to some extent, including the formula of automatically determining the number of clusters, spectral clustering of mutants, and selection of representative mutants;
- (3) The effectiveness of the mutant reduction strategies are evaluated through an empirical study based on 12 real-life programs.

The rest of this paper is organized as follows. The fundamental knowledge is introduced in Section 2. Section 3 describes the proposed mutant reduction method, including the definition and calculation of distance of mutants, the determination of the number of clusters,

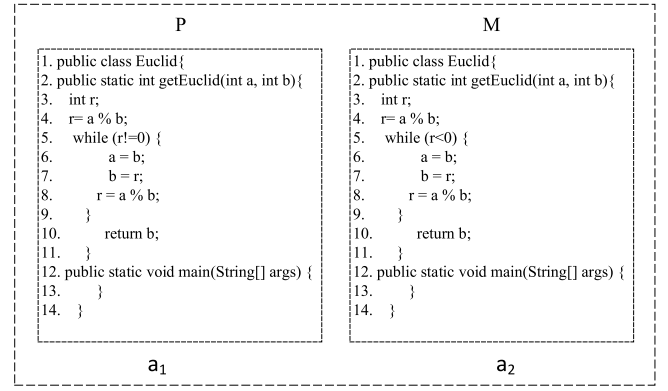


Fig. 1. A simple Java language source program *P* and one mutant *M*.

and the clustering technique. The design and setting of our empirical study are presented in Section 4. Section 5 gives and analyzes the experimental results. The work related to our study is discussed in Section 6. Finally, Section 7 summarizes the paper and identifies the future work.

2. Background

In this study, spectral clustering is used to improve the efficacy of mutation testing. The background information of mutation testing and clustering techniques are introduced in this section.

2.1. Mutation testing

Given a base program *P*, mutation testing applies the so-called mutation operator to seed some grammatical change, and thus generates a faulty version, termed as a mutant *M*. For example, the left side of Fig. 1 is a simple Java program that finds the greatest common divisor of two integers. By changing the statement “while (r!=0)” to “while (r<0)” in the source program, a mutant is obtained, as shown in the right side of Fig. 1.

The source program *P* and the mutant *M* can be executed by a test datum *t*, separately. If their final outputs are different, the mutant *M* is deemed to be killed by *t*. For example, for test datum $t_1 = \{a = 2, b = 3\}$, the outputs of *P* and *M* shown as Fig. 1 are 1 and 3, respectively. Thus, *M* is said to be killed by t_1 , implying that the corresponding seeded fault is detected. Given another test case $t_2 = \{a = 3, b = 3\}$, the outputs of *P* and *M* are the same, in which case, *M* is not killed by t_2 .

Killing a mutant requires three conditions: (1) The test datum must be able to execute the mutated statement; (2) After executing the mutated statement, the state of the program must change; and (3) The changed state of the program must affect the output of the program. DeMillo et al. [22] defined the above three conditions for killing mutants as *accessibility*, *necessity* and *adequacy*, respectively. Note that satisfying all three conditions is also called the *strong mutation*, while the *weak mutation* only requires the first two conditions.

The strong mutation testing needs to execute the whole program, while in the weak mutation testing, the codes after the mutant statement does not need to be executed. Previous studies have shown that weak mutation testing have almost the same defect detectability as strong mutation testing while saving 50% execution cost [23,24]. Therefore, the weak mutation testing is often used instead of the strong one.

In order to reduce the execution cost of weak mutation testing further, However, Papadakis and Malevris [25,26] structured mutant branches by combining the original and mutant statement into a branch statement, called *mutant branch*, and then formed a new program, denoted as *P'*, by integrating all these mutant branches into the original

program. If a test datum can cover the true branch of a mutant branch in P' , it will kill the corresponding mutant according to the criterion of weak mutation testing. Therefore, the problem of weakly killing mutants of P is transformed into that of covering the true branches of all mutant branches in P' .

There exist some mutants that always show the same execution behavior as the base program. In other words, they cannot be killed by any test datum. They are called *equivalent mutants* [27,28]. For the example program P in Fig. 1, if “ $b = r$ ” on Line 7 is changed to “ $b = r + +$ ”, the resultant mutant actually always gives the same output as P , and thus is an equivalent mutant to P .

We normally evaluate the quality of a test suite T by the mutation score, which is defined as: $MS_T = N_k / (N_M - N_E)$, where MS_T is the mutation score of T , N_k is the number of mutants killed by T , N_M is the total number of mutants, and N_E is the number of equivalent mutants.

2.2. Spectral clustering

Spectral Clustering (SC) [29] is a clustering algorithm based on spectral partitioning, which intends to transform the clustering problem into an optimal segmentation problem of graphs.

Let $\Pi = \{\pi_1, \pi_2, \dots, \pi_n\}$ represent a set, in which π_i is the i th individual. The purpose of clustering is to divide all individuals into several categories, so that the individuals within the same category are similar to each other according to some criteria. In spectral clustering, we can firstly obtain the similarity matrix A by some given measurement formula. Then, according to A , a weighted graph G is constructed, denoted as $G(V, E)$, where the elements in the vertex set V represent the individuals to be clustered, and E is the set of edges in G . Each edge of G has a weight ω , which refers to the similarity between vertices. The problem of clustering individuals can be transformed into that of dividing a graph into several subgraphs, so that the weight of each subgraph is the largest [30]. The commonly used partition criterion are minimum cut set [31], normalized cut set [32], and so on. For minimum cut set criterion, the minimum cut can be expressed as: $mincut(G) = \min\{cut(A, B)\}$, where $cut(A, B) = \sum_{\mu \in A, \nu \in B} \omega(\mu, \nu)$, $\omega(\mu, \nu)$ is the weight between μ and ν , and A and B are all subsets of V which satisfy $A \cup B = V$ and $A \cap B = \emptyset$. For normalized cut set criterion, we can also express minimum normalized cut with the following formula: $minnormalizedcut(G) = \min\{\frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}\}$, where $assoc(A, V) = \sum_{\mu \in A, \nu \in V} \omega(\mu, \nu)$ and $assoc(B, V) = \sum_{\mu \in B, \nu \in V} \omega(\mu, \nu)$. However, such a problem is NP-hard according to the partitioning criterion. One viable solution is to consider the continuous relaxation form of the original problem [33,34]. In this way, the clustering problem can be transformed into the spectral decomposition of Laplacian matrix (Similarity matrix). Laplacian matrix is also called affinity matrix, denoted as A . One expression of A can be defined as follows [35]: $A = (a_{ij})_{n \times n}$, where

$$a_{ij} = \begin{cases} \exp\left\{-\frac{d_{ij}^2}{2h^2}\right\} & \text{if } i \neq j \\ 0 & \text{if } i = j, \end{cases}$$

and d_{ij} represents the distance between the i th individual and the j th individual (Euclidean distance or Hamming distance), h represents the Gaussian kernel parameter [32], which determines the attenuation speed between individuals. Here, h can be determined according to the method proposed in Ref. [35].

Before the formula of Laplacian matrix is given, we first give the definition of degree matrix, which is represented by the symbol $R = (R_{ij})_{n \times n}$. R is a diagonal matrix and the diagonal elements can be expressed as $R_{ii} = \sum_{j=1}^n a_{ij}$, where $a_{ij} \in A$, $i = 1, 2, \dots, n$. When $i \neq j$, $R_{ij} = 0$. In different spectral clustering algorithms, the used formula of Laplacian matrix is different. For example, unnormalized Laplacian matrix L ($L = R - A$), normalized Laplacian matrix [35] H_{sym} ($H_{sym} = R^{-\frac{1}{2}} \cdot A \cdot R^{-\frac{1}{2}}$ or $H_{sym} = R^{-\frac{1}{2}} \cdot A$) and so on. In this paper, we adopt the normalized Laplacian matrix $H_{sym} = R^{-\frac{1}{2}} \cdot A \cdot R^{-\frac{1}{2}}$.

Considering the continuous relaxation form of the problem, we can transform the optimization problem of normalized cut set criteria into the following eigenvalue decomposition problem: $Ay = (1 - \lambda)Ry$. In the same way, Let $z = R^{-\frac{1}{2}}y$, we can transform the problem into the following eigenvalue decomposition problem: $H_{sym}z = R^{-\frac{1}{2}} \cdot A \cdot R^{-\frac{1}{2}}z = (1 - \lambda)z$.

Finally, we use the method proposed in [35] to extract and decompose the eigenvalues of the normalized Laplacian matrix (H_{sym}), and then select the eigenvectors corresponding to the first κ maximum eigenvalues to form R^κ space. Here, every point in R^κ corresponds to an original individual, and clustering algorithm (e.g., K -means) can be applied in R^κ space. According to the front eigenvector of H_{sym} , we can get the approximate solution of the problem. Generally, the approximation of the minimum cut of undirected weighted graph can be achieved by using the leading eigenvector information of Laplacian matrix [32,36]. In this way, we can map the original clustering problem to that of clustering the eigenvectors corresponding to the first K ($K = \kappa$) eigenvalues, so as to achieve the purpose of dimensionality reduction. More importantly, as compared with other similar clustering algorithms, spectral clustering algorithm is not only simple and easy to implement, but also can determine the number of clusters adaptively without adding other parameters.

All in all, spectral clustering has different specific implementation methods according to different criteria functions, but the overall process can be summarized into the following three steps:

- **Step 1:** Construct the similarity matrix (Laplacian matrix) Y of individuals;
- **Step 2:** Obtain the first K eigenvalues and extract the eigenvectors corresponding to the first K eigenvalues from Y to construct a new data feature space;
- **Step 3:** Use classical clustering algorithms such as K -means clustering algorithm [12] or agglomerative clustering algorithm (ACA) [12] to cluster individuals in eigenvector space, and the clustering results are mapped back to the original space.

From the above steps we can see that, different from the classical clustering algorithm, spectral clustering algorithm is used to cluster individuals in eigenvector space, and the clustering results are mapped back to the original data space in step 3. More importantly, step 3 will make the algorithm more adaptive to the sample space and also overcomes the limitation of sample space in traditional clustering algorithms.

As compared with other typical clustering analysis algorithms, spectral clustering algorithm does not depend on the shape of sample space in the process of clustering. In other words, traditional clustering algorithms are most based on convex spherical sample space. When the sample space is not convex, they will fall into local optimum. Spectral clustering can achieve clustering in arbitrary shape sample space and convergence to the global optimal solution [35]. The proposed mutant reduction technique that is based on the spectral clustering can automatically determine the number of clusters, and thus improve the scalability across different scales of programs. At the same time, spectral clustering algorithm can effectively overcome the shortcomings of some classical clustering algorithms that are easy to converge to the local optimal solution, and can get the global optimal solution. Spectral clustering algorithm was first used in computer vision [37], VLSI design [38] and other fields, and recently was used in machine learning [32], and quickly became a research hotspot in the field of machine learning in the world. Up to now, there is no literature about applying spectral clustering algorithm to mutation testing. In this paper, we improve spectral clustering and apply it to mutation testing.

3. Mutant reduction based on spectral clustering

In this section, we first introduce some definitions as the basis for our methodology. Then, we present a mutant reduction method based on spectral clustering.

```

M1: AOIS_008
(Line4 in Fig.1(a1)→r=a%b--)
M2: AOIU_001
(Line4 in Fig.1(a1)→r=-a%b)
M3: AOIU_003
(Line7 in Fig.1(a1)→b=-r)
M4: AORB_002
(Line4 in Fig.1(a1)→r=a/b)
M5: LOI_001
(Line4 in Fig.1(a1)→r=~a%b)
M6: ROR_003
(Line5 in Fig.1(a1)→r<0)

```

Fig. 2. Six mutants of the example program P .

```

t1: {1, 3}   t2: {5, 5}   t3: {3, 5}   t4: {1, 5}
t5: {3, 2}   t6: {3, 4}   t7: {4, 4}   t8: {4, 5}

```

Fig. 3. Eight test cases of the example program P .

3.1. Definitions

In order to use spectral clustering method to reduce mutants, we first need to give the similarity or distance between mutants. We hope that the mutant in the same cluster will have the same error detection ability. That is to say, for two mutants, the closer the test data can kill them, the more similar the two mutants should be. Therefore, we give the following definition:

Definition 1 (Similarity of Mutants). Suppose that D is the input domain of program P , M_i and M_j are two mutants, and D_i and D_j are two sub-domains of D in which the test data can kill M_i and M_j , respectively. Then the similarity of M_i and M_j is defined as:

$$sim(M_i, M_j) = \frac{|D_i \cap D_j| + |\bar{D}_i \cap \bar{D}_j|}{|D|}$$

where $|\cdot|$ represents the size of the region. Each datum in $D_i \cap D_j$ can kill both M_i and M_j , and each datum in $\bar{D}_i \cap \bar{D}_j$ can kill neither M_i nor M_j . When $D_i = D_j$, $sim(M_i, M_j) = 1$.

In the clustering process, we mainly use the distance between mutants, so we give the following definition:

Definition 2 (Distance of Mutants). The distance of M_i and M_j is defined as:

$$dis(M_i, M_j) = 1 - sim(M_i, M_j) = 1 - \frac{|D_i \cap D_j| + |\bar{D}_i \cap \bar{D}_j|}{|D|} = \frac{|D_i \cap \bar{D}_j| + |\bar{D}_i \cap D_j|}{|D|} \quad (1)$$

But generally, D_i and D_j are unknown. Therefore, we propose a statistical method to estimate the distance between mutants, which requires to sample the domain.

Definition 3 (Killing Matrix of Mutation Testing). Given a test suite $T = \{t_1, t_2, \dots, t_m\}$ and a set of mutants $\Omega = \{M_1, M_2, \dots, M_n\}$, we can

obtain a matrix according to the killing results of mutation testing:

$$\mathbf{K} = \begin{bmatrix} k_{11} & k_{12} & \dots & k_{1m} \\ k_{21} & k_{22} & \dots & k_{2m} \\ \dots & \dots & \dots & \dots \\ k_{n1} & k_{n2} & \dots & k_{nm} \end{bmatrix}$$

where

$$k_{ij} = \begin{cases} 1 & \text{if } M_i \text{ is killed by } t_j \\ 0 & \text{otherwise} \end{cases}$$

and $i = 1, 2, \dots, n, j = 1, 2, \dots, m$.

Test suite $T = \{t_1, t_2, \dots, t_m\}$ can be regarded as a sample of size m from the input domain D , so we can replace D with m in formula (1). In addition, $|D_i \cap \bar{D}_j|$ can be replaced by the number of test data that can kill M_i but cannot kill M_j and $|\bar{D}_i \cap D_j|$ can be replaced by the number of test data that cannot kill M_i but can kill M_j . Therefore, we can obtain the following approximate calculation formula of distance:

$$dis(M_i, M_j) \approx \frac{\sum_{l=1}^m d_{ij}^l}{m} \quad (2)$$

where

$$d_{ij}^l = \begin{cases} 0 & \text{if } k_{il} = k_{jl} \\ 1 & \text{if } k_{il} \neq k_{jl} \end{cases}$$

and $i, j = 1, 2, \dots, n, i \neq j, l = 1, 2, \dots, m$.

With the increase of sample size m , $\frac{\sum_{l=1}^m d_{ij}^l}{m}$ will approach $dis(M_i, M_j)$ more and more. In particular, if $T = D$, $dis(M_i, M_j) = \frac{\sum_{l=1}^m d_{ij}^l}{m}$. Therefore, the approximate formula given in this paper is reasonable.

It should be noted that here mutation testing can refer to strong mutation testing or weak mutation testing. In order to distinguish the killing matrices obtained under the two mutation criteria, we denote them as $\mathbf{K}_{\text{strong}}$ and $\mathbf{K}_{\text{weak}}$, respectively. Because strong mutation testing is time-consuming to execute the whole program, we can judge whether the mutant is killed or not under the criterion of weak mutation testing. Previous studies have shown that most of the mutants that can be killed under the criterion of weak mutation testing can also be killed under the criterion of strong mutation testing. Therefore, the killing matrix obtained by the two methods should be very similar. In the weak mutation testing, this paper uses the idea of Ref. [39] to transform the problem of weakly killing mutants into that of covering mutation branches, and then insert all mutant branches into the original program to obtain a transformed program. In this way, for each test datum, we can get the killing results of all mutants by just executing the transformed program once. If the strong mutation testing is used, in order to get the killing results of executing n mutants with m test cases, it is necessary to execute the mutants $m \times n$ times. Now, with the present method, we only need to run the transformed program n times. Therefore, through the improvement, we greatly reduce the computing cost.

Note that there exist some mutation testing tools, such as *MuJava* [21] and *MuClipse* [20,40], that can help us automatically generate the $\mathbf{K}_{\text{strong}}$ and $\mathbf{K}_{\text{weak}}$. Therefore, the calculation of killing matrix can be done automatically.

Now, we will take the program in Fig. 1 as an example to illustrate the above concepts and calculations. Without the loss of generality, further assume that there are six mutants ($M_1, M_2, \dots, M_6$) generated from P after applying some mutation operators, shown as Fig. 2. In addition, there exist eight test cases ($t_1, t_2, \dots, t_8$) for testing P , shown as Fig. 3.

To get $\mathbf{K}_{\text{weak}}$, the specific steps can be explained as follows: (1) Obtain the mutant branches corresponding to the above six mutants by the weak mutation transformation method, shown as Fig. 4; (2) Insert the six mutant branches into the source program P to obtain the new program P' , shown as Fig. 5, in which the bold part represents the


```

M1B: if((a % b) != (a % (b - 1)))
M2B: if((a % b) != (-a % b))
M3B: if(r != -r)
M4B: if((a % b) != (a / b))
M5B: if((a % b) != (~a % b))
M6B: if((r != 0) != (r < 0))

```

Fig. 4. Six mutant branches of the example program P .

```

1. public class Euclid{
//public static int[] A= new int[6];
2. public static int getEuclid(int a,
int b){
3.     int r;
if((a % b) != (a % (b-1))) {A[0]=1;} //M1B
if((a % b) != (-a % b)) {A[1]=1;} //M2B
if((a % b) != (a / b)) {A[3]=1;} //M4B
if((a % b) != (~a % b)) {A[4]=1;} //M5B
4.     r = a % b;
if((r != 0) != (r < 0)) {A[5]=1;} //M6B
5.     while (r!=0) {
6.         a = b;
if(r != -r) {A[2]=1;} //M3B
7.         b = r;
8.         r = a % b;
9.     }
10.    return b;
11. }
12. public static void
main(String[] args) {
13. }
14. }

```

Fig. 5. A new program P' obtained by inserting the example program P .

inserted mutant branches; (3) Execute the new program P' with eight test data to obtain the following matrix:

$$K_{\text{weak}} = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

When performing weak mutation testing, for each test datum, we can get the killing results of all mutants by just executing the transformed program once. For this example, we need to execute the transformed program 8 times. But for strong mutation testing, we need to execute each mutant 8 times, so the total number of executing mutants is $6 \times 8 = 48$, which is 6 times as much as that of weak mutation testing.

We can further calculate distance between each pair of mutants according to formula (2). For example, the distance between M_1 and

M_2 can be calculated as follows:

$$dis(M_1, M_2) = \frac{\sum_{l=1}^8 d_{12}^l}{8} = \frac{5}{8} = 0.625$$

For convenience, all distances can be expressed in the form of a matrix:

Definition 4 (Distance Matrix). Given the distance between each pair of mutants, we can construct a matrix: $\mathbf{D} = (d_{ij})_{n \times n}$, where $d_{ij} = dis(M_i, M_j)$, $i, j = 1, \dots, n$.

According to Definition 4, we can get the distance matrix for the above example as follows:

$$\mathbf{D} = \begin{bmatrix} 0 & 0.625 & 0.625 & 0.500 & 0.375 & 0.625 \\ 0.625 & 0 & 0 & 0.375 & 0.250 & 0 \\ 0.625 & 0 & 0 & 0.375 & 0.250 & 0 \\ 0.500 & 0.375 & 0.375 & 0 & 0.125 & 0.375 \\ 0.375 & 0.250 & 0.250 & 0.125 & 0 & 0.250 \\ 0.625 & 0 & 0 & 0.375 & 0.250 & 0 \end{bmatrix}$$

Once we get the distance matrix of mutants, we will give the method of spectral clustering.

3.2. Mutant reduction method based on the spectral clustering

We propose a mutant reduction method based on the spectral clustering, abbreviated as SCMT. It consists of three major steps: (1) the automatic determination of the number of clusters; (2) the clustering of mutants based on spectral clustering; and (3) the selection of mutants from clusters. It should be noted that the data obtained in this part are calculated by MTLAB (2012a) software, and the three digit valid data are reserved for non-zero numbers.

3.2.1. Automatically determining the number of clusters

At present, there are many methods to automatically determine the number of clusters, and the focus of this paper is to determine the number of clusters by the difference between eigenvalues [41–43]. Of course, a lot of researchers have done some work in this field. For example, Sun et al. [44] used the difference between the eigenvalues of the Laplacian matrix to determine the number of classes. The basic formula in their method is $g_i = \lambda_i - \lambda_{i+1}$, where $\lambda_i \geq \lambda_{i+1}$, $i = 1, 2, 3, \dots, n-1$. The number of clusters can be decided through the calculation $K = \arg \max_i \{g_i\}$, where $\phi = \{g_1, g_2, \dots, g_{n-1}\}$, Lu et al. [45] used the interval formula between the eigenvalues of the Laplacian matrix to determine the number of classes. The basic formula of their method is $z_i = e^{\lambda_{i+1}} - e^{\lambda_i}$, where $\lambda_i \leq \lambda_{i+1}$, $i = 1, 2, 3, \dots, n-1$. The number of clusters can be decided through the calculation $K = \arg \max_i \{z_i\}$, where $\phi = \{z_1, z_2, \dots, z_{n-1}\}$.

Although the above methods can automatically determine the number of clusters, but they all affect both the accuracy and scalability of clustering. In order to address this issue, we propose the negative growth rate of the eigenvalue interval ($B_i = \Delta \delta_i / e^{\lambda_i}$, where $\Delta \delta_i = e^{\lambda_{i+1}} - e^{\lambda_i}$, $\lambda_i \geq \lambda_{i+1}$ ($\lambda_j \in \mathbf{V}_{n \times n}$, $j = 1, 2, 3, \dots, n$), $i = 1, 2, 3, \dots, n-1$) that automatically determining the number of clusters based on eigenvalues and eigenvectors.

The main steps for determining the number of clusters are as follows:

- **Step 1:** Given the distance matrix $\mathbf{D}$ of order n , re-order the elements in each row in ascending sequence to get a new matrix $\mathbf{D}'$. Obtain the number η_i such that $\{D'_{i, \eta_i+1} - D'_{i, \eta_i}\} = \max_{1 \leq j \leq n-1} \{D'_{i, j+1} - D'_{i, j}\}$, where $i = 1, 2, \dots, n$.

For the above example, we can get $\mathbf{D}'$ and $\eta_i=2$ by **Steps 1**, where $i = 1$, $\mathbf{D}'$ is as follows:

$$\mathbf{D}' = \begin{bmatrix} 0 & 0.375 & 0.500 & 0.625 & 0.625 & 0.625 \\ 0 & 0 & 0 & 0.250 & 0.375 & 0.625 \\ 0 & 0 & 0 & 0.250 & 0.375 & 0.625 \\ 0 & 0.125 & 0.375 & 0.375 & 0.375 & 0.500 \\ 0 & 0.125 & 0.250 & 0.250 & 0.250 & 0.375 \\ 0 & 0 & 0 & 0.250 & 0.375 & 0.625 \end{bmatrix}$$

- **Step 2:** The Gaussian kernel parameter h can be calculated according to the formula $\exp\left\{-\frac{\nabla^2}{2h^2}\right\} = \epsilon$, where ϵ is a very small positive real number, $\nabla = \max_{1 \leq i \leq n} \{D'_{i, \eta_i}\}$.

- **Step 3:** For a given distance matrix $\mathbf{D}$, calculate the weighted adjacency matrix $\mathbf{A} = (a_{ij})_{n \times n}$ that is also called the similarity matrix, where $a_{ij} \in \mathbf{A}$ ($i = j = 1, 2, \dots, n$) represents the degree of similarity, that is

$$a_{ij} = \begin{cases} \exp\left\{-\frac{d_{ij}^2}{2h^2}\right\} & \text{if } i \neq j \\ 0 & \text{if } i = j. \end{cases}$$

For the example, we can get $h = 0.08$ by **Steps 1** and **2**. On this basis, the following weighted adjacency matrix is constructed by **Step 3**:

$$\mathbf{A} = \begin{bmatrix} 0 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 \\ 0.000 & 0 & 1.000 & 0.000 & 0.008 & 1.000 \\ 0.000 & 1.000 & 0 & 0.000 & 0.008 & 1.000 \\ 0.000 & 0.000 & 0.000 & 0 & 0.295 & 0.000 \\ 0.000 & 0.008 & 0.008 & 0.295 & 0 & 0.008 \\ 0.000 & 1.000 & 1.000 & 0.000 & 0.008 & 0 \end{bmatrix}$$

- **Step 4:** Construct a normalized Laplacian matrix $\mathbf{H}_{sym}$, that is, $\mathbf{H}_{sym} = \mathbf{R}^{-\frac{1}{2}} \cdot \mathbf{A} \cdot \mathbf{R}^{-\frac{1}{2}}$, where $\mathbf{R}$ is the degree matrix of $\mathbf{A}$. Each element in $\mathbf{R}$ is

$$r_{ij} = \begin{cases} \sum_{j_0=1}^n a_{ij_0} & \text{if } i = j \\ 0 & \text{if } i \neq j. \end{cases}$$

For the example, according to **Step 4**, the degree matrix of $\mathbf{A}$ is as follows:

$$\mathbf{R} = \begin{bmatrix} 0.000 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2.008 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2.008 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.295 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.319 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2.008 \end{bmatrix}$$

Then, the normalized Laplacian matrix is

$$\mathbf{H}_{sym} = \mathbf{R}^{-\frac{1}{2}} \cdot \mathbf{A} \cdot \mathbf{R}^{-\frac{1}{2}} = \begin{bmatrix} 0 & 0.000 & 0.000 & 0.000 & 0.007 & 0.000 \\ 0.000 & 0 & 0.498 & 0.000 & 0.009 & 0.498 \\ 0.000 & 0.498 & 0 & 0.000 & 0.009 & 0.498 \\ 0.000 & 0.000 & 0.000 & 0 & 0.963 & 0.000 \\ 0.008 & 0.009 & 0.009 & 0.963 & 0 & 0.009 \\ 0.000 & 0.498 & 0.498 & 0.000 & 0.009 & 0 \end{bmatrix}$$

- **Step 5:** For $\mathbf{H}_{sym}$, calculate its associated eigenvalues and eigenvectors, which are stored in two $n \times n$ matrices $\mathbf{G}$ and $\mathbf{U}$, respectively. Then, arrange the eigenvalues in ascending order to get a diagonal matrix $\mathbf{V}_{n \times n}$, where the eigenvalues satisfy $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_n$, and the eigenvectors corresponding to the eigenvalues are denoted as $c_1, c_2, \dots, c_n$.

For the example, according to **Step 5**, we obtain the eigenvalues of $\mathbf{H}_{sym}$ and their corresponding eigenvectors, denoted by matrices $\mathbf{G}$ and

$\mathbf{U}$, respectively.

$$\mathbf{G} = \begin{bmatrix} -0.000 & 0 & 0 & 0 & 0 & 0 \\ 0 & -0.964 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.960 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.000 & 0 & 0 \\ 0 & 0 & 0 & 0 & -0.498 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.498 \end{bmatrix}$$

$$\mathbf{U} = \begin{bmatrix} -1.000 & -0.005 & -0.005 & -0.002 & -0.000 & -0.000 \\ -0.000 & -0.003 & 0.175 & -0.550 & 0.384 & 0.816 \\ -0.000 & -0.003 & 0.175 & -0.550 & -0.816 & -0.390 \\ 0.008 & -0.707 & -0.675 & -0.211 & 0.000 & 0.000 \\ -0.000 & 0.707 & -0.672 & -0.219 & 0.000 & -0.000 \\ -0.000 & -0.003 & 0.175 & -0.550 & 0.432 & -0.426 \end{bmatrix}$$

Furthermore, the eigenvalues are arranged in descending order, the result of which is represented by matrix $\mathbf{V}$:

$$\mathbf{V} = \begin{bmatrix} 1.000 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.960 & 0 & 0 & 0 & 0 \\ 0 & 0 & -0.000 & 0 & 0 & 0 \\ 0 & 0 & 0 & -0.498 & 0 & 0 \\ 0 & 0 & 0 & 0 & -0.498 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.964 \end{bmatrix}$$

- **Step 6:** The negative growth rate of the eigenvalue interval is calculated by formula $B_i = \Delta\delta_i / e^{\lambda_i}$, where $\Delta\delta_i = e^{\lambda_{i+1}} - e^{\lambda_i}$, $\lambda_i \geq \lambda_{i+1}$ ($\lambda_j \in \mathbf{V}_{n \times n}, j = 1, 2, 3, \dots, n$), $i = 1, 2, 3, \dots, n-1$.

Based on the results of **Step 5** and following **Step 6**, we obtain the negative growth rate of the eigenvalue interval for the example, as shown in the following matrix $\mathbf{I}$:

$$\mathbf{I} = \begin{bmatrix} -0.039 \\ -0.617 \\ -0.392 \\ -0.000 \\ -0.372 \end{bmatrix}$$

- **Step 7:** Calculate the number of clusters $K = \arg_{i \in \Gamma}^{min} \{I\}$, where $\Gamma = \{B_1, B_2, \dots, B_{n-1}\}$.

For the example, from the results of **Step 6** and the formula of **Step 7**, the subscript with the negative growth rate of the minimum eigenvalue interval is 2, that means the number of clusters $K = 2$.

It should be noted that there are different ways to determine the distance between mutants. For example, Hussain [12] and Zeng et al. [17] used the killing ability to determine the distance of the mutants that is also known as the execution-based method. However, the method determines the distance by executing all mutants repeatedly against all test cases. In order to reduce the impact of implementing all mutants, a domain analysis was used by Ji et al. [46] to determine distance among mutants. Although this method could reduce the cost of executing all mutants, its effectiveness is subordinate to the symbolic execution's ability. In a different study, a graph edit distance and convolution kernel were used by Strug et al. [47] to determine distance among mutants. In the paper, we use only one method that is the execution-based method to determine the distance among mutants, whereas the investigations of other methods will be our future work.

3.2.2. Spectral clustering of mutants

The following steps are taken to cluster mutants according to an adaptive spectral clustering technique.

- **Step 8:** According to **Step 5**, extract the eigenvectors corresponding to the first K maximum eigenvalues in the $n \times n$ matrix $\mathbf{U}$ to form an $n \times K$ feature matrix $\mathbf{Q}_{n \times K}$, where each column vector represents an eigenvector in $\mathbf{U}$ has a one-to-one correspondence with each non-zero element that represents an eigenvalue in $\mathbf{G}$.

As an illustration, refer to the above example again. Given $K = 2$ that is the result of **Step 7**, we can get the following feature matrix by **Step 8**:

$$\mathbf{Q}_{6 \times 2} = \begin{bmatrix} -0.002 & -0.005 \\ -0.550 & 0.175 \\ -0.550 & 0.175 \\ -0.211 & -0.675 \\ -0.219 & -0.672 \\ -0.550 & 0.175 \end{bmatrix}$$

- **Step 9:** The K -means clustering algorithm is used to cluster $\mathbf{Q}_{n \times K}$ whose each row is regarded as a point to obtain a n -dimensional vector $\mathbf{O}$, where O_i represents the i th element in $\mathbf{O}$ and $O_i \in \{1, 2, \dots, K\}$ ($i = 1, 2, \dots, n$).

For the example, in **Step 9**, the matrix $\mathbf{Q}_{6 \times 2}$ is clustered by K -means algorithm, and thus we can obtain the following vector:

$$\mathbf{O} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 2 \\ 2 \\ 1 \end{bmatrix}$$

- **Step 10:** Construct a clustering set $\mathbf{Z} = \{C_1, C_2, \dots, C_K\}$, where C_j ($j = 1, 2, \dots, K$) is the cluster of mutants $\{M_{j_1}, M_{j_2}, \dots, M_{j_h}\}$ such that $O_{j_1} = O_{j_2} = \dots = O_{j_h} = j$, where O_{j_t} represents the t th element in the j th class in $\mathbf{O}$, $t = 1, 2, \dots, h$. In other words, all the mutants with the same corresponding O_i will be put into the same cluster.

According to **Step 10**, we get the clustering results $\mathbf{Z} = \{C_1, C_2\} = \{\{M_1, M_2, M_3, M_6\}, \{M_4, M_5\}\}$ for the above example. In other words, there are two clusters C_1 and C_2 , among which C_1 contains M_1, M_2, M_3 and M_6 , and C_2 contains M_4 and M_5 .

It should be noted that different from other clustering algorithms, spectral clustering algorithm does not directly cluster the original individuals, but extracts the eigenvectors corresponding to the first K largest eigenvalues of Laplacian matrix that is denoted by $\mathbf{H}_{\text{sym}}$ in this paper. The clustering problem of original individuals is transformed into the clustering problem of eigenvector space that is denoted by $\mathbf{U}$ in this paper corresponding to the first K largest eigenvalues of $\mathbf{H}_{\text{sym}}$.

3.2.3. Selection of mutants

Mutants in each cluster are regarded as similar to one another. In this study, we construct the reduced subset of mutants by simply selecting one mutant from each cluster in a random manner, that is:

- **Step 11:** Randomly select one mutant from each C_j ($j = 1, 2, \dots, K$).

Refer to the above example. Given the clustering result $\mathbf{Z} = \{C_1, C_2\} = \{\{M_1, M_2, M_3, M_6\}, \{M_4, M_5\}\}$, one possible reduced subset can be $\{M_1, M_5\}$. By the killing matrix $\mathbf{K}_{\text{weak}}$ we can see that M_1 and M_5 can kill all these 6 mutants. In fact, M_1 and M_5 can kill all mutant even under the criterion of strong mutation testing. The above results fully show that the spectral clustering method for mutants proposed in this paper is reasonable.

It should be noted that we conjecture that spectral clustering is able to put mutants with the similar features into the same cluster, that is, the mutants in each cluster are regarded as having similar features, including code structure, etc. Thus, we use the random selection method to choose representative mutants, that is, the probability of each mutant being selected in each cluster is equal. More importantly, the random selection of a mutant from a cluster already gave a good performance, which was confirmed by other studies [12,17,18]. Some in-depth investigation of code structure, etc. will help improve the effectiveness and thus will be our future work.

4. Empirical study

We conduct an empirical study to evaluate the effectiveness and applicability of the proposed mutant reduction method. The experimental settings are presented in the following, including the research questions, variables and measures, object programs and mutant generation, experimental procedure and threats to validity.

4.1. Research questions

Our experiments are designed to answer the following three research questions:

RQ1: Can the proposed SCMT method reduce the number of mutants without jeopardizing the effectiveness of mutation testing?

The reason why RQ1 is set up is to verify effectiveness of reducing the number of mutants by used the proposed method (SCMT) in this paper. Assume that M_{total} is the set of all mutants and M_{reduced} ($M_{\text{reduced}} \subseteq M_{\text{total}}$) is a reduced subset of mutants. Further assume that a test suite T can kill all mutants in M_{reduced} . In the ideal case, it is expected that T can also kill all non-equivalent mutants in M_{total} , which implies that M_{reduced} is a sufficient representative subset for M_{total} . In this study, we attempt to examine whether SCMT is able to significantly reduce the mutant set while preserving the similar effectiveness with the original mutant set. During the experiment, we also compare the three methods from the time cost of calculating the distance between mutants and the time cost of the whole experiment process. In addition, in order to analyze the above experimental results more scientifically, the statistical software SPSS is used to T-statistics detection of SCMT and other five benchmark methods to determine whether there is significant difference.

RQ2: What is the performance of SCMT by comparing with other similar clustering techniques that automatically determine the number of clusters?

The reason why RQ2 is set up is to verify performance of the SCMT method in this paper. In other words, as compared with the other three baseline clustering techniques, the mechanism of automatical determining the number of clusters, our method (SCMT) can achieve an outstanding balance between the efficiency that is represented by the size of reduced mutant subset and the effectiveness in preserving the high mutation score.

One major process of SCMT is to automatically determine the number of clusters, and the number of clusters will become the size of the reduced mutant subset. In this study, we would evaluate and compare the performance of SCMT with other benchmark techniques in automatically determining the number of clusters in terms of both mutant reduction performance and mutant killing capability. In addition, we compare the four methods from the time cost of the whole experiment process during the experiment.

RQ3: How effective is the spectral clustering of mutants for mutation testing?

The reason why RQ3 is set up is to verify that the spectral clustering of mutants for mutation testing is effective. We will evaluate and compare the performance of SCMT with other benchmark techniques. The spectral clustering of mutants is another major process in SCMT. In this study, we compare the effectiveness of SCMT with those of other benchmark clustering algorithms. In addition, we also compare the six methods from the time cost of the whole experiment process during the experiment.

4.2. Variables and measures

4.2.1. Independent variables

The independent variable in our study is the mutant reduction strategy. Three methods are selected for answering RQ1. Although we use the weak mutation testing to reduce the cost of calculating the distance matrix of mutants, we still want to make a comparison with the strong mutation testing. In order to distinguish, the SCMT based on weak and strong mutation testing are denoted as $SCMT_{weak}$ and $SCMT_{strong}$, respectively. The two methods are identical, but different criteria are used to determine whether the mutant is killed. In addition, we also choose a benchmark clustering technique for comparison, namely Ji's method of the domain analysis [46].

For RQ2, we select three techniques that are also able to determine the number of clusters automatically. These techniques, namely Sun's method [44], Lu's method [45], and Ji's method, have similar steps to our method while using different formulas. Similar to RQ1, different mutation testing strategies are used when using our method, namely $SCMT_{weak}$ and $SCMT_{strong}$. It should be noted that these five methods are firstly applied to mutation testing in this paper.

For RQ3, five benchmark techniques, namely agglomerative clustering algorithm (ACA) [12], random sampling (RS) [13], K -means [12], the mutant reduction method based on genetic algorithm (GA) [17] and Ji's method [46], are selected as the baselines.

Agglomerative clustering algorithm (ACA) starts with a different cluster assigned to each individual. Clusters are continuously merged until the termination condition is satisfied [48]. K -means clustering algorithm is one of the first, simple and speedy clustering algorithms, but this algorithm depends on the initial choice of cluster number to a large extent [49]. SCMT in this paper is different from these two methods, which can automatically determine the number of clusters, and indirectly cluster the individuals through feature vectors. Ji's method [46] is a mutation reduction method based on domain analysis. The main idea is to first calculate the distance between mutants based on domain analysis, and then a method similar to K -means is used to reduce mutants. Random sampling method is a mutant reduction method proposed by Acree [13]. The main idea is to randomly select some mutants from a large number of mutants for mutation testing, and the rest are not considered. GA is a mutant reduction method based on genetic algorithm for clustering proposed by Zeng et al. [17]. This method uses the global search ability of genetic algorithm to adaptively generate the optimal or nearly optimal number of clusters.

4.2.2. Dependent variables

The dependent variables in this study are mainly the metrics to be used for measuring the performance of mutant reduction techniques. Mutation score (as defined in Section 2) is a fundamental metric in mutation testing. We normally evaluate the quality of a test suite T by this indicator.

However, a recent article [50] pointed out that because of the existence of so-called subsumed mutants, the use of mutation scores for evaluation may lead to imprecise conclusions. In order to eliminate the impact caused by this problem, this study considers two aspects:

(1) The focus of the paper, namely mutant reduction, will reduce the impact of the so-called subsumed mutants. More importantly, the use of spectral clustering will group similar mutants that are including the subsumed ones together, and the selection of one mutant from one cluster will definitely remove lots of subsumed mutants, and thus their influence on the experimental results can be largely reduced.

(2) The evaluation metric used in our study is not a simple mutation score of any test suite on any mutant set. In this study, to accurately measure the effectiveness of mutant reduction, we implemented the following procedure. Suppose that a mutant reduction technique is applied to reduce the original mutant set M_{total} to the subset $M_{reduced}$. A test suite T is constructed such that the mutation score of T on $M_{reduced}$ is 100%. Then, we calculate the mutation score of T on M_{total} ,

abbreviated as MS_T^{total} . Apparently, the higher value MS_T^{total} has, the more representative $M_{reduced}$ is to M_{total} . If $MS_T^{total} = 1$, we can say that the mutant reduction does not jeopardize the effectiveness of mutation testing at all. It should be noted that we used the metric MS_T^{total} , which refers to the mutation score of a test suite T on the original mutant set M_{total} , where T can achieve a 100% mutation score on the reduced mutant set $M_{reduced}$. Such a metric has fully considered the impact of mutant subsumption: similar mutants that are including subsumed ones have been removed in $M_{reduced}$, and the comparison between the performance on $M_{reduced}$ and M_{total} showed that our mutant reduction technique does not affect the overall effectiveness of mutation testing after the removal of similar mutants.

In addition to MS_T^{total} , we used another metric, namely mutant reduction rate $R_{reduction}$, in RQ1 and RQ2. $R_{reduction}$ is defined as:

$$R_{reduction} = \frac{|M_{total}| - |M_{reduced}|}{|M_{total}|} \times 100\% \quad (3)$$

where $|\cdot|$ refers to the size of a set. Intuitively speaking, the higher $R_{reduction}$ is, the fewer mutants are selected. A "good" mutant reduction technique should have a high $R_{reduction}$ while preserving MS_T^{total} as close to 1 as possible.

Finally, in evaluating questions RQ2 and RQ3, we also compare the time spent in the experimental process between the proposed method and other benchmark methods to evaluate the cost of the proposed method across 20 experimental runs.

4.3. Object programs and mutant generation

We selected 12 Java programs as the subjects of our experiments. They are from various application domains and with different sizes. The basic information of them is given in the first 4 columns of Table 1. Among them, five programs (J1 to J5) are small-sized programs that implement basic scientific calculations [25,51], while the other six programs (J6 to J12) are some popular utilities [52], which can be downloaded from <http://commons.apache.org>.

We applied two commonly used mutation testing tool, namely Muclipse [20], MuJava's plug-in on Eclipse [21], to generate mutants for all 12 Java programs. Although there are 15 method-level operators and 28 class-level operators in Muclipse, only 15 method-level mutation operators are considered in this paper. For the small-sized programs (J1-J5), we selected the whole methods for the mutant construction, while for the large-sized programs (J6-J12), we selected a part of methods for the mutant construction to control the scale of our experiments. It should be noted that not because other methods are not feasible, but considering that these methods used in this paper are easier to achieve 100% mutation score in mutation testing.

For each object program, we generated a large amount of random test cases using Randoop [53], an automated JUnit test case generation tool. Besides, for some hard to kill mutants, we would manually design test cases according to the characteristics of the program. It should be noted that Randoop is not pure random testing. It acted on Java and .Net through feedback-directed random testing. In a word, Randoop [53] considered different scenarios for achieving high coverage and thus can help us achieve 100% mutation score. In addition, Randoop cannot guarantee that all the mutants will be killed in the first place. Thus, in the process of generating test data for killing mutants, we firstly used Randoop to generate a certain number of test data to execute the mutants, and then for those that are difficult to kill, we manually generated more test data to ensure that all nonequivalent mutants are killed. After executing these test cases on all mutants, we manually examined the still-alive mutants and identified the equivalent mutants. The information about mutant generation is given in the 5th to 8th columns of Table 1.

Table 1
Object programs and mutants.

ID	Program	LOCs	Description	# Methods		# Mutants	
				Total	Mutated	Total	Equivalent
J1	TrashAndTakeOut	30	Basic arithmetic operations	2	2	111	29
J2	Mid	26	Return the middle value of three integers	1	1	115	18
J3	FourBalls	28	Relative weight of four spheres	1	1	213	49
J4	Triangle	36	Return the type of a triangle with three integer inputs	1	1	325	40
J5	Cal	46	Calculate the days between two dates in the same year	2	2	314	43
J6	HelpFormatter	416	A formatter of help messages for the current command line options	39	2	301	34
J7	WordUtils	173	Operations on Strings that contain words	12	2	243	24
J8	NumberUtils	636	Provides extra functionality for Java Number classes	47	18	404	104
J9	Dfp	1702	Decimal floating point library for Java	113	3	800	94
J10	FastMath	2311	Faster, more accurate, portable alternative to Math and StrictMath for large scale computation	100	2	1380	198
J11	ArrayUtils	3258	Operations on arrays, primitive arrays (like int[]) and primitive wrapper arrays	319	10	1798	185
J12	Colt	31 407	Project for high performance scientific and technical computing	3106	812	93 985	12 749

4.4. Experimental procedure

4.4.1. Experimental step

Our experiments are conducted on a machine with Intel(R)Core(TM)i5-3230M CPU @ 2.60 GHz, 4.00 GB ROM, Windows 8 Operating System, Eclipse SDK 4.2.2 with *Muclipse*1.3 and *MATLAB*(2012a).

In the experiments, each mutant reduction strategy is applied to obtain $M_{reduced}$ from M_{total} . The required test cases are automatically generated by Randoop [53], which can help achieve a 100% mutation score on $M_{reduced}$. In particular, a greedy algorithm [34] in this study is used to reduce the number of test data in the process of using Randoop [53] to generate random test data and we only needed to execute each mutant once with each test datum, which is different from the workload of executing the mutants repeatedly to generate test data. More importantly, in order to achieve a 100% mutation score as much as possible, we have carried out equivalence tests on some unkillable mutants. If they are equivalent mutants, we would delete them, otherwise we would keep them. In addition, in order to reduce the impact of the number of test cases on the cost of mutation testing and the computational cost of the method proposed in this paper, we also made use of the greedy algorithm [34] to minimize the test suite: Only the test case that can kill at least one alive mutant would be added into the test suite. Once the test suite is constructed, it would be applied to M_{total} to calculate MS_T^{total} . Such a process would be repeated for 20 times to get a mean value of MS_T^{total} .

Note that the test cases (or subset of test cases) used in the experiment are sampled by the greedy algorithm [34] during the process of generating all the test cases by Randoop [53], which made the test data set still have high redundancy and the quality is not very high. In addition, although greedy algorithm [34] used in this paper can reduce the number of test data, it still needed to execute all mutants. For sure, there are other ways to reduce the cost of this process. For example, Pacheco et al. [54] proposed a technique to select a small part from a large number of test inputs that might reveal software faults under test and introduced a generation-guided test input generation technology, which used the classification based on the operation model to construct legal input. Moreover, Patrick et al. [55] used evolutionary strategy and subset selection to describe and evaluate search-based techniques to identify areas of input fields that were called subdomains, so that test cases randomly sampled from these regions can be used effectively to find faults. Hao et al. [56] proposed a test data generation technique based on static defect detection, which helped identify a set of suspicious statements that might contain errors, and then test data generation issues were converted into generating test data that covered these suspicious statements.

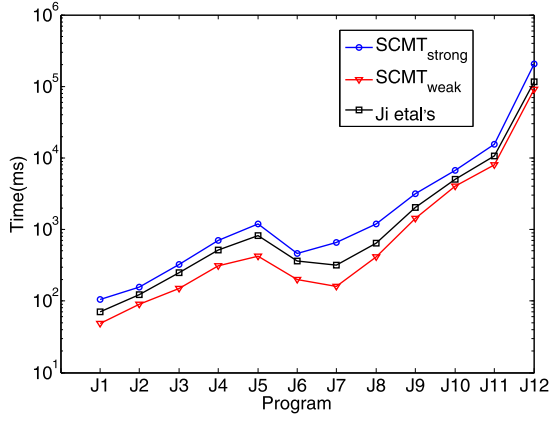
4.4.2. Evaluation procedure and metrics

In order to evaluate RQ1, we applied the method of this article to 12 typical tested programs, and each tested program can be repeated for

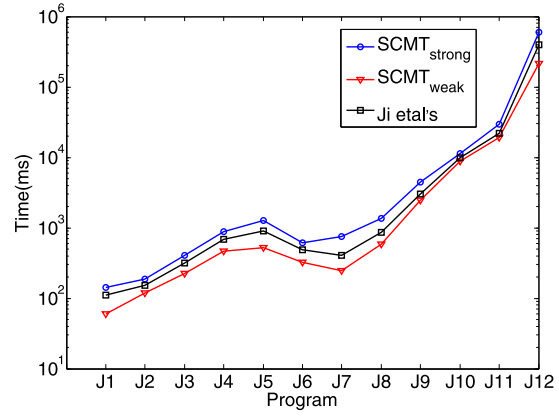
20 times to get a mean value of MS_T^{total} and $R_{reduction}$. In this study, we attempted to examine whether SCMT is able to significantly reduce the mutant set while preserving the similar effectiveness with the original mutant set. We compare different methods in terms of two indicators (MS_T^{total} and $R_{reduction}$) and the time cost in the process of experiment, including the time cost of calculating the distance between mutants and the time cost of the whole experiment process. In addition, in order to analyze the above experimental results more scientifically, the statistical software SPSS is used to conduct T-test on different methods. It should be noted that each of the four methods (*K*-means, ACA, RS and GA) can be further divided into two forms according to the strong mutation and weak mutation. For example, *K*-means can be divided into *K*-means (strong) and *K*-means (weak), and others are similar.

In order to evaluate RQ2, in this study, we would evaluate and compare the performance of SCMT ($SCMT_{weak}$ and $SCMT_{strong}$) with other three benchmark techniques in automatically determining the number of clusters (Ji's method [46], Sun's method [44] and Lu's method [45]) in terms of both mutant reduction performance ($R_{reduction}$) and mutant killing capability (MS_T^{total}). We applied the four methods of this article to 12 typical test programs, and each clustering technique on each tested program can be repeated for 20 times to get a mean value of MS_T^{total} and $R_{reduction}$. In addition, in the process of evaluating RQ2, we compare the average execution time consumed by SCMT and three benchmark methods (Ji's method [46], Sun's [44] and Lu's [45]) across 20 experimental runs. It should be noted that each of the two methods (Sun's method [44] and Lu's method [45]) can be further divided into two forms according to the strong mutation killing matrix and weak mutation killing matrix. For example, Sun's method [44] can be divided into Sun's (strong) and Sun's (weak), and the other is similar.

For RQ3, in this study, we would evaluate the performance of SCMT ($SCMT_{weak}$ and $SCMT_{strong}$) by comparing other five benchmark techniques (Ji's method [46], the *K*-means and agglomerative clustering algorithm (ACA) [12], random sampling (RS) [13], GA [17]) in terms of mutant killing capability (MS_T^{total}). It should be noted that each of the four methods (ACA, *K*-means, RS and GA) in the process of evaluating RQ3 can be further divided into two forms according to the strong mutation killing matrix and weak mutation killing matrix. For example, ACA can be divided into ACA (strong) and ACA (weak), and others are similar. We applied the six methods of this article to 12 typical test programs, and each clustering technique on each tested program can be repeated for 20 times to get a mean value of MS_T^{total} . In the evaluation process, the number of clusters in the four comparison methods is the same as the number of clusters generated by SCMT ($SCMT_{weak}$ and $SCMT_{strong}$). In addition, other parameter settings are involved for the GA method, including population size N_{size} , threshold T_0 , etc., as shown in Table 2. Especially, in the process of evaluating RQ3, we also compare the average execution time consumed by SCMT ($SCMT_{weak}$ and $SCMT_{strong}$) and five benchmark methods (*K*-means [12], ACA [12], RS [13], Ji's method [46] and GA [17]) across 20 experimental runs. It should be noted that each of the four methods (*K*-means, ACA, RS



(a) Time of getting distances for the three methods



(b) Time of performing the whole process for the three methods

Fig. 6. Comparison of the time-cost for different methods in mutant reduction.

Table 2

Parameters of GA in this paper.

Parameters	Parameter values in this paper
N_{size}	30
T_0	0.001
H	1
N	10
pm	0.05
pc	0.3

and GA) can be further divided into two forms according to the strong mutation killing matrix and weak mutation killing matrix. For example, K -means can be divided into K -means (strong) and K -means (weak), and others are similar.

Note that to ensure some form of generalization in the process of evaluating the three questions (RQ1, RQ2 and RQ3), this study has considered the following two aspects: (1) We used Randoop [53] to generate random test cases to guarantee the independence and avoid bias. (2) To further assure the generalization, we run the experiments 20 times for each program to get the statistically reliable results.

In addition, note that in order to eliminate the influence of the randomness in test case generation and mutant selection, 20 experiments are performed on each program, and our observation is that the randomness had low impact on the experimental results.

4.5. Threats to validity

4.5.1. Threat to internal validity

The main threat to internal validity is the effective implementation of the mutant reduction method proposed in this paper, which required a certain degree of programming work, including the generation of killing matrix in Section 3. With the increasing number of mutants, the method proposed in this paper would be affected by the storage and calculation cost of relevant matrix (such as K and D). In order to avoid the impact of this problem, we took into account the symmetry of these matrices when programming, and only calculated half of the matrix, thereby reducing the time consumption caused by calculating the matrix. In addition, The source code for implementing all mutant reduction techniques had been cross-checked by different individuals to minimize the error in programming.

4.5.2. Threat to external validity

The major threat to external validity is concerned with the generalization of our results. In this study, although we have selected programs with different sizes and from distinct fields, we still cannot say that the

obtained conclusions can be generalized to any type of program. The mutant generation process is also limited to some selected methods, which might also affect the generality of the study.

4.5.3. Threat to construct validity

The threat of construct validity is related to the measurement. The metric MS_T^{total} is based on the mutation score, the most fundamental metric for mutation testing. More importantly, MS_T^{total} intuitively measured the effectiveness of a mutant reduction technique. The other metric $R_{reduction}$, by nature, directly measured to what extent the mutant set can be reduced.

4.5.4. Threat to conclusion validity

Due to the randomness of some settings (e.g., the random selection of mutants from clusters), we implemented each mutant reduction technique for at least 20 times to obtain the mean values of MS_T^{total} . Since the collected data consistently showed very small standard deviations, we are confident that the results are statistically reliable.

5. Experimental results

5.1. Answer to RQ1: Effectiveness of mutant reduction

Table 3 gave the $R_{reduction}$ and the mean value of MS_T^{total} on each program for three methods ($SCMT_{weak}$, $SCMT_{strong}$ and Ji's). From Table 3, we can get the following results: (1) Over 74% mutants are removed from the original mutant set for each program in three methods. For example, for $SCMT_{weak}$ and $SCMT_{strong}$, 11 out of 12 cases exceeded 80%, and for Ji's, 10 out of 12 cases exceeded 80%. (2) MS_T^{total} is consistently very close to 100% for three methods. For example, $MS_T^{total} = 100\%$ for five programs under $SCMT_{strong}$, $MS_T^{total} = 100\%$ for four programs under $SCMT_{weak}$, and $MS_T^{total} = 100\%$ for three programs under Ji's. (3) It is clearly shown that SCMT ($SCMT_{weak}$ and $SCMT_{strong}$) can significantly reduce the mutant set without jeopardizing the effectiveness of mutation testing.

In order to further evaluate the effectiveness of SCMT, we also obtained the time cost of calculating the distance between mutants and the time cost of the whole experiment process (Fig. 6(a) and (b)). From Fig. 6, we can conclude that $SCMT_{weak}$ is better than the other two methods. In other words, the time of calculating the distance between the mutants (Fig. 6(a)) and the total execution time of the experiment process (Fig. 6(b)) for $SCMT_{weak}$ are significantly lower than the other two methods.

In addition, in order to analyze the above experimental results more scientifically, the statistical software SPSS is used to perform T-test

Table 3 $R_{reduction}$ and MS_T^{total} of SCMT and Ji's method.

ID	Program	$SCMT_{strong}$			$SCMT_{weak}$			Ji's		
		$ M_{reduced} $	$R_{reduction}(\%)$	$MS_T^{total}(\%)$	$ M_{reduced} $	$R_{reduction}(\%)$	$MS_T^{total}(\%)$	$ M_{reduced} $	$R_{reduction}(\%)$	$MS_T^{total}(\%)$
J1	TrashAndTakeOut	14	82.92	100	16	80.49	100	20	75.61	100
J2	Mid	13	86.60	100	13	86.60	100	19	80.41	100
J3	FourBalls	41	75.00	98.83	42	74.39	97.56	41	75.00	97.53
J4	Triangle	52	81.75	98.51	52	81.75	97.54	49	82.81	97.78
J5	Cal	30	88.93	100	33	87.82	99.26	38	85.98	100
J6	HelpFormatter	32	88.01	98.83	32	88.01	97.38	28	89.51	96.83
J7	WordUtils	19	91.32	100	20	90.87	100	21	90.41	97
J8	NumberUtils	54	82.00	96.84	54	82.00	97.67	48	84.00	95.34
J9	Dfp	62	91.22	98.27	62	91.22	98.31	59	91.64	97.57
J10	FastMath	65	94.50	100	63	94.67	100	70	94.08	97.66
J11	ArrayUtils	61	96.22	96.37	63	96.09	97.79	57	96.47	94.68
J12	Colt	1875	97.69	95.66	1885	97.68	96.19	1786	97.80	93.23

Table 4

Comparison of total execution time for SCMT and other benchmark technologies that automatically determining the number of clusters (Unit: ms).

Program	SCMT		Sun's		Lu's		Ji's
	$SCMT_{strong}$	$SCMT_{weak}$	Sun's (strong)	Sun's (weak)	Lu's (strong)	Lu's (weak)	
J1	144.0	59.9	148.5	91.3	150.1	92.8	112.0
J2	185.9	119.1	179.2	114.0	185.1	119.9	152.5
J3	402.8	224.0	368.5	194.7	378.2	204.3	314.4
J4	878.3	470.3	795.9	397.9	795.6	397.6	684.3
J5	1282.4	517.6	1132.3	355.6	1144.0	367.2	910.0
J6	612.3	319.9	588.6	321.3	589.3	322.0	491.9
J7	760.0	248.8	751.3	247.0	753.3	249.0	402.4
J8	1367.6	583.9	1302.5	519.4	1312.5	529.4	856.2
J9	4469.8	2480.4	4270.9	2548.3	4239.1	2516.4	3000.3
J10	11 335.0	8727.1	11 124.8	8521.2	11 080.2	8476.7	9917.8
J11	29 255.1	19 201.8	27 251.8	19 687.6	27 221.4	19 657.2	21 868.4
J12	594 550.1	213 010.5	373 411.4	297 571.9	373 012.3	297 472.2	399 584.6
Sum.	645 243.3	245 963.3	421 325.7	330 570.2	420 861.1	330 404.7	438 294.8

Table 5

Comparison of total execution time for SCMT and other clustering benchmark technologies (Unit: ms).

Program	SCMT		K-means		ACA		RS		GA		Ji's
	$SCMT_{strong}$	$SCMT_{weak}$	K-means (strong)	K-means (weak)	ACA (strong)	ACA (weak)	RS (strong)	RS (weak)	GA (strong)	GA (weak)	
J1	144.0	59.9	124.9	67.6	133.6	76.3	105.3	48.1	376.1	318.8	112.0
J2	185.9	119.1	138.5	73.3	160.0	94.9	125.3	60.1	401.0	335.8	152.5
J3	402.8	224.0	287.7	113.8	339.4	165.6	250.3	76.4	2112.4	1938.5	314.4
J4	878.3	470.3	557.8	159.8	720.3	322.3	509.9	111.9	4295.5	3897.5	684.3
J5	1282.4	517.6	969.1	192.3	1078.1	301.3	944.5	167.8	2470.3	1693.5	910.0
J6	612.3	319.9	371.2	103.9	501.1	233.7	336.2	68.8	2012.6	1745.3	491.9
J7	760.0	248.8	619.3	115.0	692.5	188.2	581.6	77.3	1296.6	792.3	402.4
J8	1367.6	583.9	1017.3	234.3	1212.4	429.3	970.5	187.5	4706.9	3923.9	856.2
J9	4469.8	2480.4	2132.2	409.6	3319.5	1596.9	2019.4	296.8	9346.9	7624.3	3000.3
J10	11 335.0	8727.1	3184.1	580.5	6569.9	3966.4	3083.1	479.6	13 634.9	11 031.4	9917.8
J11	29 255.1	19 201.8	8892.3	1328.0	15 514.2	7950.0	8622.6	1058.4	36 762.6	29 198.3	21 868.4
J12	594 550.1	213 010.5	97 824.6	22 188.5	244 130.7	88 391.0	95 110.4	20 489.3	656 621.0	480 930.4	399 584.6
Sum.	645 243.3	245 963.3	116 119.0	25 566.6	274 371.7	103 715.9	112 659.1	23 122.0	734 036.8	543 430.0	438 294.8

on SCMT ($SCMT_{weak}$ and $SCMT_{strong}$) and Ji's method to determine whether there is significant difference.

Firstly, we qualitatively described some statistics of the three methods including the methods in this paper, as shown in Table 6. From Table 6, we can get the following results: (1) The minimum value of the average mutation score obtained by $SCMT_{weak}$ is 96.19%, which is higher than the minimum value of the other two methods. (2) Three methods ($SCMT_{weak}$, $SCMT_{strong}$ and Ji's) can achieve the maximum value that is average mutation score of 100%. (3) The mean value of $SCMT_{strong}$ is 98.61%, which is higher than other two methods. (4) The standard deviation and variance of $SCMT_{weak}$ are 1.32 and 1.75, respectively. In this way, average mutation scores obtained by $SCMT_{weak}$ are more stable. (5) According to the skewness and kurtosis, we can conclude that every method obeyed the normal distribution approximately. We can see that $SCMT_{weak}$ is better than the other methods from these statistics.

Then, SPSS software is used to conduct T-test on SCMT ($SCMT_{strong}$ and $SCMT_{weak}$) and Ji's method, and the test results are shown in Table 7. From Table 7, we can draw the following conclusions: (1) There is a significant difference between SCMT ($SCMT_{weak}$ or $SCMT_{strong}$) and Ji's method (p -value is less than 0.05). (2) There is no significant difference in mutation scores between $SCMT_{weak}$ and $SCMT_{strong}$ (p -value is higher than 0.05), which indicated that $SCMT_{weak}$ can replace $SCMT_{strong}$ under certain conditions.

It should be noted that there may be subsumption relationship between the mutants in the reduction set. The reduction methods of mutants based on clustering can reduce the number of mutants to a large extent, and cannot guarantee a minimum set. However, by the experimental results, the number of mutants obtained by our method is the lowest. In addition, for most mutant reduction methods based on clustering, after clustering the mutants, an individual is randomly selected as the representative one. Therefore, we adopt the same method. Our experimental results consistently told us that even the randomly

Table 6

Description statistics of SCMT and other benchmark methods.

Methods	Number of samples	Minimum (%)	Maximum (%)	Sum (%)	Mean value (%)	Standard deviation	Variance	Skewness		Kurtosis	
								Skewness value	Standard error	Kurtosis value	Standard error
$SCMT_{strong}$	12	95.66	100.00	1183.31	98.61	1.56	2.43	-0.83	0.64	-0.58	1.23
$SCMT_{weak}$	12	96.19	100.00	1181.70	98.48	1.32	1.75	-0.05	0.64	-1.30	1.23
Ji's	12	93.23	100.00	1166.62	97.22	2.13	4.54	-0.25	0.64	-0.31	1.23
Sun's (strong)	12	5.99	96.72	881.46	73.46	24.48	599.16	-2.11	0.64	5.37	1.23
Lu's (strong)	12	11.60	90.77	806.33	67.19	22.11	488.92	-1.54	0.64	2.83	1.23
Sun's (weak)	12	20.99	97.52	912.98	76.08	21.03	442.29	-1.80	0.64	3.82	1.23
Lu's (weak)	12	31.95	92.62	872.10	72.68	18.56	344.50	-1.01	0.64	0.55	1.23
K-means (strong)	12	91.22	99.85	1158.86	96.57	2.69	7.26	-0.89	0.64	0.04	1.23
ACA (strong)	12	90.59	100.00	1146.25	95.52	3.25	10.56	-0.27	0.64	-1.41	1.23
RS (strong)	12	86.55	91.98	1075.59	89.63	2.05	4.19	-0.40	0.64	-1.25	1.23
GA (strong)	12	93.52	99.51	1165.58	97.13	1.92	3.70	-0.79	0.64	-0.18	1.23
K-means (weak)	12	90.12	99.76	1152.16	96.01	3.02	9.12	-0.93	0.64	-0.21	1.23
ACA (weak)	12	90.43	100.00	1145.24	95.44	3.38	11.40	-0.33	0.64	-1.36	1.23
RS (weak)	12	85.67	91.96	1072.35	89.36	1.97	3.87	-0.56	0.64	-0.38	1.23
GA (weak)	12	92.77	99.51	1164.84	97.07	1.99	3.95	-1.38	0.64	1.29	1.23

Table 7

T-test results of SCMT and other benchmark techniques.

Method of comparison		Mean value of difference	Standard deviation of difference	Standard error of mean value	95% confidence interval of difference		p (Bilateral)
					Lower limit	Upper limit	
$SCMT_{strong}$	to $SCMT_{weak}$	0.134	0.852	0.246	-0.407	0.675	0.596
$SCMT_{strong}$	to Ji's	1.391	1.020	0.294	0.743	2.039	0.001
$SCMT_{strong}$	to Sun's (strong)	25.154	24.037	6.939	9.882	40.426	0.004
$SCMT_{strong}$	to Lu's (strong)	31.415	21.870	6.313	17.520	45.311	0.000
$SCMT_{strong}$	to K-means (strong)	2.038	1.492	0.431	1.089	2.986	0.001
$SCMT_{strong}$	to ACA (strong)	3.088	2.088	0.603	1.762	4.415	0.000
$SCMT_{strong}$	to RS (strong)	8.977	1.831	0.529	7.813	10.140	0.000
$SCMT_{strong}$	to GA (strong)	1.478	0.784	0.226	0.979	1.976	0.000
$SCMT_{strong}$	to Sun's (weak)	22.527	20.586	5.943	9.447	35.607	0.003
$SCMT_{strong}$	to Lu's (weak)	25.934	18.182	5.249	14.382	37.487	0.000
$SCMT_{strong}$	to K-means (weak)	2.596	1.833	0.529	1.431	3.760	0.000
$SCMT_{strong}$	to ACA (weak)	3.173	2.157	0.623	1.802	4.543	0.000
$SCMT_{strong}$	to RS (weak)	9.247	2.117	0.611	7.902	10.592	0.000
$SCMT_{strong}$	to GA (weak)	1.539	0.931	0.269	0.947	2.131	0.000
$SCMT_{weak}$	to Ji's	1.257	1.394	0.402	0.371	2.142	0.010
$SCMT_{weak}$	to Sun's (strong)	25.020	24.195	6.985	9.647	40.393	0.004
$SCMT_{weak}$	to Lu's (strong)	31.281	21.968	6.341	17.324	45.239	0.000
$SCMT_{weak}$	to K-means (strong)	1.903	2.079	0.600	0.582	3.224	0.009
$SCMT_{weak}$	to ACA (strong)	2.954	2.323	0.671	1.478	4.430	0.001
$SCMT_{weak}$	to RS (strong)	8.843	2.213	0.639	7.437	10.248	0.000
$SCMT_{weak}$	to GA (strong)	1.343	1.451	0.419	0.422	2.265	0.008
$SCMT_{weak}$	to Sun's (weak)	22.393	20.699	5.975	9.242	35.544	0.003
$SCMT_{weak}$	to Lu's (weak)	25.800	18.184	5.249	14.246	37.354	0.000
$SCMT_{weak}$	to K-means (weak)	2.462	2.362	0.682	0.961	3.962	0.004
$SCMT_{weak}$	to ACA (weak)	3.038	2.411	0.696	1.507	4.570	0.001
$SCMT_{weak}$	to RS (weak)	9.113	2.194	0.633	7.718	10.507	0.000
$SCMT_{weak}$	to GA (weak)	1.405	1.618	0.467	0.377	2.433	0.012

selected mutants can collectively achieve a thorough coverage of mutants, e.g., the killing capabilities of test cases as shown in Table 3, Figs. 7 and 8. In other words, even if random selection is used to select mutants from different clusters, and the randomly selected subset of mutants can cohesively represent the whole set of mutants.

5.2. Answer to RQ2: Automatic determination of the number of clusters

Fig. 7 presented the results of comparison between SCMT and the three baseline techniques of determining the number of clusters (Ji's, Sun's and Lu's).

Although SCMT ($SCMT_{weak}$ and $SCMT_{strong}$) had lower $R_{reduction}$ than the other three methods (Fig. 7(b) and (d)), its MS_T^{total} (Fig. 7(a) and (c)) is always much higher, close to 100%. More specifically, as can be seen from Fig. 7(b), Sun's (strong) and Lu's (strong) had higher $R_{reduction}$ (for 11 out of 12 cases, over 90%) and thus had lower MS_T^{total} (Fig. 7(a)) values (for 1 out of 12 cases, over 95%) than $SCMT_{strong}$. In other words, as compare with the other three baseline techniques of

automatically determining the number of clusters, our method (SCMT) can achieve an outstanding balance between the efficiency that is represented by the size of the reduced mutant subset and the effectiveness in preserving the high mutation score.

In order to analyze the above experimental results more scientifically, the statistical software SPSS is used to perform T-test on SCMT ($SCMT_{weak}$ and $SCMT_{strong}$) and other three methods (Ji's, Sun's and Lu's) to determine whether there is significant difference.

Firstly, we qualitatively described some statistics of the six methods including the methods in this paper, as shown in Table 6. From Table 6, we can get the following results: (1) The minimum value of the average mutation score obtained by $SCMT_{weak}$ is 96.19%, which is higher than the minimum value of the other three methods. Note that each of the two methods (Sun's and Lu's) can be further divided into two forms according to the strong mutation and weak mutation. For example, the Sun's can be divided into Sun's (strong) and Sun's (weak), and the other is similar. (2) Two methods (SCMT and Ji's) can achieve the maximum value that is average mutation score of 100%, while other methods

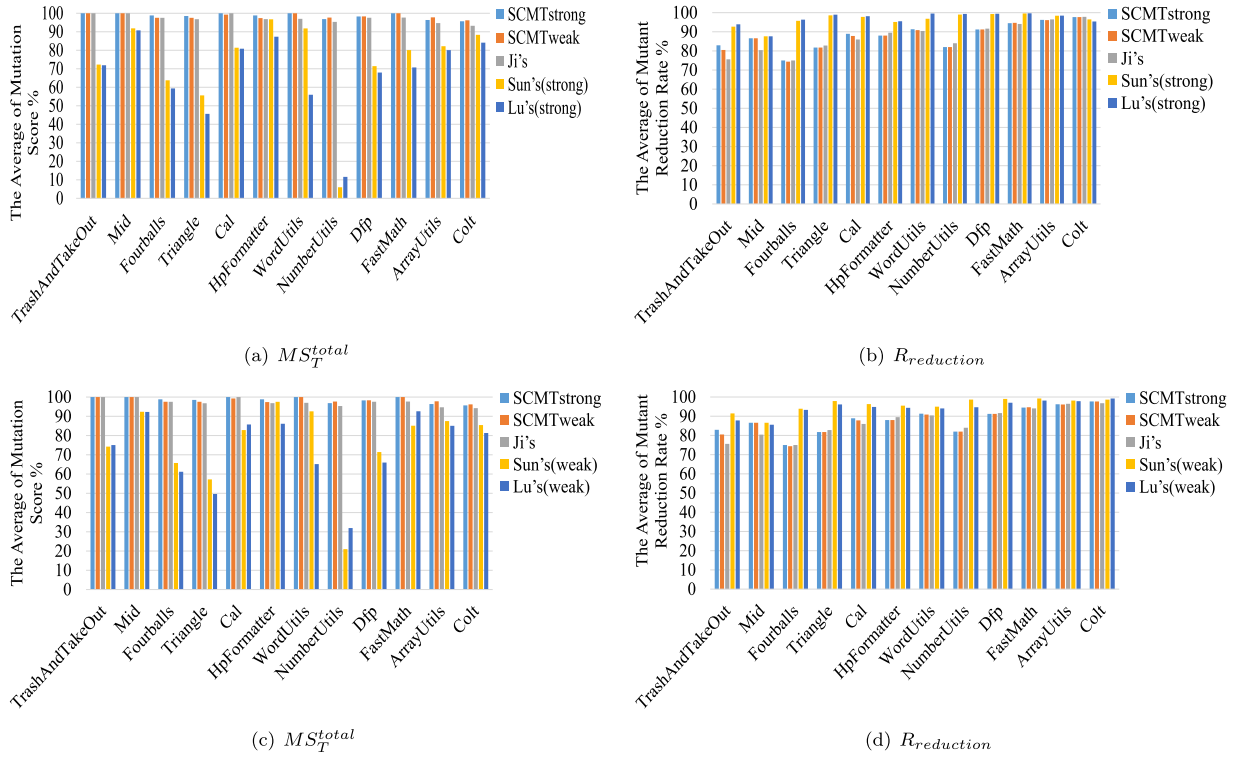
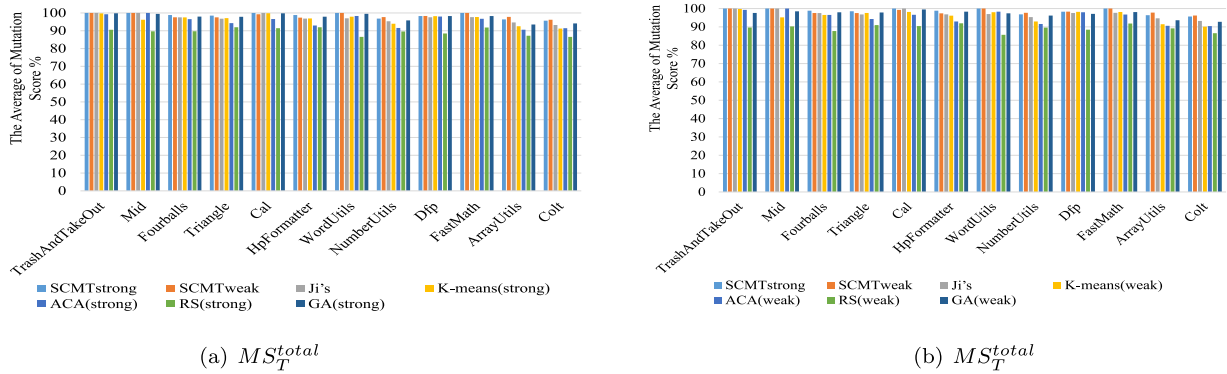


Fig. 7. Comparisons of methods for automatically determining the number of clusters.

Fig. 8. Comparison of MS_T^{total} for different benchmark techniques.

cannot. (3) The mean value of $SCMT_{strong}$ is 98.61%, which is higher than other methods, and Lu's (strong) is the lowest. (4) The standard deviation and variance of $SCMT_{weak}$ are 1.32 and 1.75, respectively. In this way, average mutation scores obtained by $SCMT_{weak}$ are more stable. (5) According to the skewness and kurtosis, we can conclude that every method obeyed the normal distribution approximately. We can see that $SCMT_{weak}$ is better than the other methods from these statistics.

Then, *SPSS* software is used to conduct T-test on SCMT ($SCMT_{strong}$ and $SCMT_{weak}$) and other three benchmark methods (Ji's, Sun's and Lu's), and the test results are shown in Table 7. Note that each of the two methods (Sun's and Lu's) can be further divided into two forms according to the strong mutation and weak mutation. For example, the Sun's can be divided into Sun's (strong) and Sun's (weak), and the other is similar. From p-value in the last column of Table 7, we can draw the following conclusions: (1) There is a significant difference between SCMT ($SCMT_{weak}$ or $SCMT_{strong}$) and the other three benchmark methods (p -value is less than 0.05). It should be noted that each of the two methods (Sun's and Lu's) can be further divided

into two forms according to the strong mutation and weak mutation. For example, the Sun's can be divided into Sun's (strong) and Sun's (weak), and the other is similar. (2) There is no significant difference in mutation scores between $SCMT_{weak}$ and $SCMT_{strong}$ (p -value is higher than 0.05), which indicated that $SCMT_{weak}$ can replace $SCMT_{strong}$ under certain conditions.

In addition, according to the characteristics of the four algorithms (SCMT, Ji's, Sun's and Lu's), we can apply them to 12 benchmark programs and obtained their average execution time across 20 experimental runs separately as shown in Table 4. We can observe that the sum of average execution time for $SCMT_{weak}$ on 12 benchmark programs is less than other methods.

5.3. Answer to RQ3: Performance of spectral clustering

The comparison results between SCMT and the other five benchmark techniques are shown as Fig. 8.

It can be seen from Fig. 8 that the average of MS_T^{total} obtained by $SCMT_{strong}$ is the highest and more than 95% for each program,

followed by the average of MS_T^{total} obtained by $SCMT_{weak}$. The average of MS_T^{total} obtained by RS (strong) is less than 90% except for J1, J4, J5, J6 and J10. In addition, although there is almost no gap between $SCMT_{weak}$ and $SCMT_{strong}$ in the average of MS_T^{total} , $SCMT_{weak}$ and $SCMT_{strong}$ are better than Ji's method in the average of MS_T^{total} . It implied that the spectral clustering used in SCMT ($SCMT_{strong}$ and $SCMT_{weak}$) can outperform four other comparison methods in the average of MS_T^{total} . It should be noted that although the outperformance looked marginal, the spectral clustering had another advantage: The number of clusters can be determined automatically using the spectrum-related concepts. Although GA can also automatically determine the number of clusters, it needed to determine other parameters, such as crossover probability and mutation probability. On the contrary, the method proposed in this paper is a self-adaptive method to determine the number of clusters without external parameters and this is not applicable to K -means and ACA, indicating that SCMT had higher efficiency and scalability.

In order to analyze the above experimental results more scientifically, the statistical software SPSS is used to perform T-test on SCMT and other five benchmark methods to determine whether there is significant difference.

Firstly, we qualitatively described some statistics of the six methods including the methods in this paper, as shown in Table 6. From Table 6, we can get the following results: (1) The minimum value of the average mutation score obtained by $SCMT_{weak}$ is 96.19%, which is higher than the minimum value of the other four methods. Note that each of the four methods (ACA, K -means, RS and GA) can be further divided into two forms according to the strong mutation and weak mutation. For example, ACA can be divided into ACA (strong) and ACA (weak), and others are similar. (2) Three methods (SCMT, Ji's and ACA) can achieve the maximum value that is average mutation score of 100%, while other methods cannot. (3) The mean value of $SCMT_{strong}$ is 98.61%, which is higher than other methods, and RS (weak) is the lowest. (4) The standard deviation and variance of $SCMT_{weak}$ are 1.32 and 1.75, respectively. In this way, average mutation scores obtained by $SCMT_{weak}$ are more stable. (5) According to the skewness and kurtosis, we can conclude that every method obeyed the normal distribution approximately. We can see that $SCMT_{weak}$ is better than the other methods from these statistics.

Then, SPSS software is used to conduct T-test on SCMT ($SCMT_{strong}$ and $SCMT_{weak}$) and other five benchmark methods (Ji's, K -means, ACA, RS and GA), and the test results are shown in Table 7. Note that each of the four methods (ACA, K -means, RS and GA) can be further divided into two forms according to the strong mutation and weak mutation. For example, ACA can be divided into ACA (strong) and ACA (weak), and others are similar. From p-value in the last column of Table 7, we can draw the following conclusions: (1) There is a significant difference between SCMT ($SCMT_{weak}$ or $SCMT_{strong}$) and other five benchmark methods (p-value is less than 0.05). It should be noted that each of the four methods (ACA, K -means, RS and GA) can be further divided into two forms according to the strong mutation and weak mutation. For example, ACA can be divided into ACA (strong) and ACA (weak), and others are similar. (2) There is no significant difference in mutation scores between $SCMT_{weak}$ and $SCMT_{strong}$ (p-value is higher than 0.05), which indicated that $SCMT_{weak}$ can replace $SCMT_{strong}$ under certain conditions.

In addition, according to the characteristics of the six algorithms (SCMT, Ji's, K -means, ACA, RS and GA), we can apply them to 12 benchmark programs and obtain their average execution time across 20 experimental runs separately as shown in Table 5. By comparing the average execution time, we can draw the following conclusions: (1) RS (weak) spent the least time in reducing mutants, while GA (strong) took the most time; (2) The total time cost of $SCMT_{weak}$ (245 963.3 ms) in reducing mutants is lower than $SCMT_{strong}$ (645 243.3 ms), Ji's (438 294.8 ms) and GA (weak) (543 430.0 ms), but the time cost of $SCMT_{weak}$ is higher than the three methods (ACA, K -means and RS).

Note that since these three methods did not automatically determine the time cost of the number of clusters, the time cost of them is much lower than other methods. (3) Although the time cost of SCMT ($SCMT_{strong}$ and $SCMT_{weak}$) in reducing mutants is not optimal, results in Table 5 can at least prove that the total execution time of four clustering techniques (K -means, ACA, RS and GA) based on weak mutation killing matrix can be greatly reduced, and other unique aspects of SCMT, e.g., automatic determination of the number of clusters and low parameter dependence, superior to other methods.

6. Related work

Mutant reduction is a major way of decreasing the cost of mutation testing. A straightforward approach for reducing the number of mutants is to randomly select part of mutants [13]. Wong and Mathur [57,58] verified the effect of the value of the mutant selection on the results of experiments. The results showed that the random sampling method had the best optimization effect when the ratio exceeded 10%. However, random sampling (RS) may cause the omission of some important mutants.

A more systematic mutant reduction approach is to select mutants based on part of mutation operators, which is first proposed by Mathur [15]. Offutt et al. [59] found that some mutants can be effectively reduced by ignoring the ASR and SVR operators during the experiment. Namin et al. [60] regarded the mutation operator selection problem as a statistical problem and adopted a linear statistical method. Twenty-eight mutation operators are selected from 108 mutation operators in C language, and it is found that 92% of mutants can be reduced while the adequacy of test cases are still guaranteed. Wong and Mathur [58] investigated all the 22 mutation operators in the mutation testing tool Mothra, and identified two typical operators, based on which highly representative mutants can be selected. Vincenzi et al. [61] proposed a family of incremental techniques to select part of mutation operators for unit and integration testing. Different from these techniques, our method does not disregard any mutation operator and thus preserves the diversity in types of seeded faults.

The application of higher-order mutation is another relatively new method to reduce the number of mutants. Jia and Harman [62,63] used a metaheuristic search method to combine multiple first-order mutants into one higher-order mutant, thereby reducing the number of mutants. The experimental results of Polo et al. [64] showed that the use of second-order mutants instead of first-order mutants can reduce the cost of mutation testing by about 50% while maintaining the high performance of mutation testing. There also exist other types of mutant reduction strategies in the literature, such as the path-aware approach to mutant reduction [65,66] and the domain reduction technique [46]. Another way to reduce the cost of mutation testing is by optimizing the execution time. King and Offutt [67], for example, converted the source program into intermediate code and then mutated and executed the intermediate code. Untch et al. [68] proposed a schema-based approach for generating one single meta-mutant of containing all mutants. In addition, in order to reduce a large number of mutants, Gong et al. [69] used domain correlation for weak mutation testing. This technology not only effectively reduced mutants, but also improved the efficiency of weak mutation testing, and then reduced the cost of mutation testing. Finally, Chekam et al. [70] used light-weight (static) features to predict representative mutants and build a mutant selection technique that identifies killable mutants that can find faults before performing any mutants. In addition, predicting mutations relied heavily on test code and dynamic features, whereas this study used only static features on the code under test and needed to build better classifiers.

Although the methods mentioned above can effectively reduce mutants and the mutation test is efficient, the similarity between mutants is not considered, which makes the reduced mutants still have certain redundancy. Therefore, some new techniques are needed to solve the problem of mutant reduction.

Considering the similarity between mutants, the use of clustering in mutation testing is first proposed by Hussain [12]. The main idea is to use the K -means and agglomerative clustering algorithm to cluster the mutants according to a similarity measure. However, the number of clusters and the initial position of the cluster center in the K -means algorithm are difficult to determine, and the process of the agglomerative clustering algorithm makes it difficult to correct the existing cluster formation. Ma et al. [19] proposed a clustering based cost reduction method especially for overlapping mutants. This method would cluster the mutants that are relatively easy to kill at the expression level in dynamic clustering, and only one mutant in each cluster would fully execute the test cases. Their experimental studies demonstrated that their approach reduced the cost of mutation testing without compromising effectiveness. Lu et al. [18] proposed a mutant reduction method based on ordering points to identify the clustering structure (OPTICS) algorithm. Zeng et al. [17] proposed a mutant reduction method based on genetic algorithm. As compare with these clustering based techniques, the proposed method does not cluster samples directly, but does it indirectly based on eigenvalues and eigenvectors. In addition, it can automatically determine the number of clusters, which improves the scalability of mutant reduction. It also has the advantage of clustering in arbitrary shape sample space and convergence to the global optimal solution.

7. Conclusions and future work

Mutation testing can be used to guide the generation of test cases that are effective in detecting various faults as well as to assess the effectiveness of a testing method. However, due to the large number of mutants that are often generated and used in mutation testing, it also incurs a high cost, which, in turn, hinders its applicability in many practical situations. In this paper, we developed a mutant reduction method based on an intelligent technique, namely spectral clustering. We gave some concepts for measurement of distance between mutants as the theoretical basis of the proposed method. In our method, the number of clusters are first automatically determined, and then the spectral clustering technique is applied to classify mutants into clusters. A subset of mutants is finally constructed by selecting one mutant from each cluster.

An experimental study is conducted to measure and compare the performance of the proposed method with other clustering based mutant reduction techniques. The experimental results showed that the proposed method can significantly reduce the number of mutants without jeopardizing the effectiveness of mutation testing. As compare with other five benchmark clustering based techniques, $SCMT_{weak}$ consistently showed the best performance. In a word, $SCMT_{weak}$ cannot only significantly reduce the number of mutants without jeopardizing the effectiveness of mutation testing, but also help significantly reduce the time cost of mutation testing and improve its applicability as well as efficiency.

It can be seen from the experiments in this paper that although the method proposed in this paper has obtained better results, we will further optimize the method proposed in this paper including the method of automatically determining the Gaussian kernel parameters. Here we can provide several reference research directions:

Firstly, the generation of the killing matrix based on weak mutation (or strong mutation) also needs to be further optimized to improve the efficiency of the mutation testing, so that the researchers can further reduce the cost of mutation testing.

Secondly, the method of calculating the distance between mutants can be used as another research direction in the future. Although three methods (Ji's, $SCMT_{strong}$ and $SCMT_{weak}$) are applied to determine the distance between mutants in this paper and $SCMT_{weak}$ can show better results and the time cost is the lowest from the experimental results, other code elements, such as statements and branches can also

be used as the characteristics for the distance calculation. It is thus our future work to conduct more in-depth investigations on this direction.

Thirdly, it is worthwhile to further investigate other methods that are different from the random one used in this initial study for selecting representative mutants.

Finally, the reduced mutants still have important value in the process of software evolution. On the one hand, the reduced mutant set, which includes both alive and killed mutants, can provide foundation for evaluation in mutation analysis. Even if there are some changes in the software, the reduced mutant set can still be used to evaluate the test methods. On the other hand, if the software change is not large, most of the mutants can still be reused. Nowadays, most software development is incremental. For example, continuous integration may evolve several times a day. In other words, incremental evolution is very common, thus most of the test cases generated by these mutants are still valid. In short, we will explore the factors influencing the reuse of the reduced mutant set in the process of software evolution in the future, so as to improve the quality of reduced mutant set.

CRedit authorship contribution statement

Changqing Wei: Acquisition, analysis, or interpretation of data, Drafted the work or revised it critically for important intellectual content. **Xiangjuan Yao:** Conception or design of the work, Acquisition, analysis, Drafted the work or revised it critically for important intellectual content, Approved the final version to be published. **Dunwei Gong:** Drafted the work or revised it critically for important intellectual content. **Huai Liu:** Drafted the work or revised it critically for important intellectual content.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the Fundamental Research Funds for the Central Universities, China (No. 2020ZDPYMS40).

References

- [1] C. Zhang, Research on Regression Test Case Selection Technology Based on Cluster Analysis (Master's thesis), Nanjing University, 2011.
- [2] R.A. DeMillo, R.J. Lipton, F.G. Sayward, Hints on test data selection: Help for the practicing programmer, *Computer* 11 (4) (1978) 34–41.
- [3] R.G. Hamlet, Testing programs with the aid of a compiler, *IEEE Trans. Softw. Eng.* 4 (1977) 279–290.
- [4] R. Just, D. Jalali, L. Inozemtseva, et al., Are Mutants a Valid Substitute for Real Faults in Software Testing? University of Washington, Washington, 2014.
- [5] B.K. Aichernig, H. Brandl, et al., Killing strategies for model-based mutation testing, *ISoftw. Test. Verif. Reliab.* 8 (4) (2014) 371–379.
- [6] I. Rubab, S. Ali, L. Briand, Y.L. Traon, Model-based testing of obligations, *Qual. Softw.(QSIC)* (2014) 1–10.
- [7] J.F. Chen, Y.S. Lu, X.D. Xie, M. Harman, Research on software fault injection testing, *J. Softw.* 20 (6) (2009) 1425–1443.
- [8] A.K. Ghani, A.K. Nasser, A.A. Jacob, Ferrari: A flexible software-based fault and error injection system, *IEEE Trans. Comput.* 44 (2) (1995) 248–260.
- [9] Q. Wang, S. Wu, M. Li, Software defect prediction technology, *J. Softw.* 19 (7) (2008) 1565–1580.
- [10] A.J. Offutt, R.H. Untch, Mutation 2000: Uniting the orthogonal, in: *Mutation Testing for the New Century*, Springer, 2001, pp. 34–44.
- [11] J.H. Andrews, L.C. Briand, Y. Labiche, Is mutation an appropriate tool for testing experiments? in: *Proceedings of the 27th International Conference on Software Engineering*, ACM, 2005.
- [12] S. Hussain, Mutation Clustering (Master's thesis), Kings College London, Strand, London, 2008.
- [13] A.T. Acree Jr, On Mutation, Tech. Rep., Georgia Inst Of Tech Atlanta School Of Information And Computer Science, 1980.

- [14] T.A. Budd, Mutation Analysis of Program Test Data (Ph.D. thesis), Yale University, 1980.
- [15] A.P. Mathur, W.E. Wong, An empirical comparison of data flow and mutation-based test adequacy criteria, *Softw. Test. Verif. Reliab.* 4 (1) (2010) 9–31.
- [16] L. Zhang, S. Hou, J. Hu, T. Xie, H. Mei, Is operator-based mutant selection superior to random mutant selection? in: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1, ACM, 2010.
- [17] F. Zeng, Y. Huang, M. Zhang, N. Pan, Mutants reduction based on genetic algorithm for clustering, *J. Comput. Appl.* 31 (5) (2011) 1314–1317.
- [18] X. Lu, S. Huang, W. Wang, Mutant reduction technique research based on optics algorithm, *J. PLA Univ. Sci. Technol. (Nat. Sci.)* (5) (2016) 101–104.
- [19] Y.S. Ma, S.W. Kim, Mutation testing cost reduction by clustering overlapped mutants, *J. Syst. Softw.* 115 (C) (2016) 18–30.
- [20] Y.S. Ma, J. Offutt, P.K. Yong, Mjjava: an automated class mutation system, *Softw. Test. Verif. Reliab.* 15 (2) (2010) 97–133.
- [21] X. Chen, Q. Gu, Mutation testing: Principal, optimization and application, *J. Front. Comput. Sci. Technol.* (2012) 1057–1075.
- [22] R.A. DeMillo, A.J. Offutt, Constraint-based automatic test data generation, *IEEE Trans. Softw. Eng.* 17 (9) (1991) 900–910.
- [23] A.J. Offutt, S.D. Lee, How strong is weak mutation? in: Proceedings of the Symposium on Testing, Analysis, and Verification, Association for Computing Machinery, New York, NY, USA, 1991.
- [24] A.J. Offutt, S.D. Lee, An empirical evaluation of weak mutation, *IEEE Trans. Softw. Eng.* 20 (5) (1994) 337–344.
- [25] M. Papadakis, N. Malevris, Automatically performing weak mutation with the aid of symbolic execution, concolic testing and search-based testing, *Softw. Qual. J.* 19 (4) (2011) 691–723.
- [26] M. Papadakis, N. Malevris, Mutation based test case generation via a path selection strategy, *Inf. Softw. Technol.* 54 (9) (2012) 915–932.
- [27] J. Lopez, N. Kushik, N. Yevtushenko, Source code optimization using equivalent mutants, *Inf. Softw. Technol.* 103 (NOV.) (2018) 138–141.
- [28] F. Wu, J. Nanavati, M. Harman, Y. Jia, J. Krinke, Mutation testing, *Inf. Softw. Technol.* 45 (9) (2018) 898–918.
- [29] X. Liu, Research on Data Mining Clustering Algorithm Based on Improved SC (Master's thesis), Lanzhou University of Technology, 2018.
- [30] X. Zhang, J. Li, H. Yu, Local density adaptive similarity measurement for spectral clustering, *Pattern Recognit. Lett.* 32 (2) (2011) 352–358.
- [31] Z. Wu, R. Leahy, An optimal graph theoretic approach to data clustering: theory and its application to image segmentation, *IEEE Trans. Pattern Anal. Mach. Intell.* 15 (11) (1993) 0–1113.
- [32] J. Shi, J. Malik, Normalized cuts and image segmentation, *IEEE Trans. Pattern Anal. Mach. Intell.* 22 (8) (2000) 888–905.
- [33] C. Zha, Y. Dou, M. Guo, D. Yue, A new hybrid clustering algorithm based on stimulated annealing, in: International Conference on Intelligent Human-Machine Systems and Cybernetics, 2013.
- [34] L. Zheng, M. Harman, R.M. Hierons, Search algorithms for regression test case prioritization, *IEEE Trans. Softw. Eng.* 33 (4) (2007) 225–237.
- [35] A.Y. Ng, M.I. Jordan, Y. Weiss, On spectral clustering: Analysis and an algorithm, in: Advances in Neural Information Processing Systems, 2002.
- [36] U. Luxburg, A tutorial on spectral clustering, *Stat. Comput.* 17 (4) (2007) 395–416.
- [37] J. Malik, S. Belongie, T. Leung, J. Shi, Contour and texture analysis for image segmentation, *Int. J. Comput. Vis.* 43 (1) (2001) 7–27.
- [38] Y. Weiss, Segmentation using eigenvectors: A unifying view, in: Proceedings of the Seventh IEEE International Conference on Computer Vision, 1999.
- [39] M. Papadakis, N. Malevris, Automatically performing weak mutation with the aid of symbolic execution, concolic testing and search-based testing, *Softw. Qual. J.* 19 (4) (2011) 691–723.
- [40] B.H. Smith, L. Williams, On guiding the augmentation of an automated test suite via mutation analysis, *Empir. Softw. Eng.* 14 (3) (2009) 341–369.
- [41] D. Vemna, M. Meila, A Comparison of Spectral Clustering Algorithm, University of Washington Seattle: Department of CSE, 2002, pp. 1–19.
- [42] X. Guo, X. Wu, Q. Song, Improved similarity parameter estimation for semi-supervised spectral clustering algorithm, *J. Northeast Dianli Univ.* 30 (6) (2010) 82–86.
- [43] M. Fiedler, Algebraic connectivity of graphs, *Czech. Math.* 23 (2) (1973) 298–305.
- [44] C. Sun, W. Kong, G. Dai, A spectral clustering with ascertainable clustering number, *J. Hangzhou Dianzi Univ.* 30 (2) (2010) 53–56.
- [45] P. Lu, Y. Cai, A spectral clustering algorithm for automatic determination of eigenvectors and category quantity, *J. Lanzhou Univ. Technol.* 44 (2) (2018) 90–94.
- [46] C. Ji, Z. Chen, B. Xu, Z. Zhao, A novel method of mutation clustering based on domain analysis, in: Software Engineering and Knowledge Engineering, 2009.
- [47] J. Strug, B. Strug, Using structural similarity to classify tests in mutation testing, *Appl. Mech. Mater.* 378 (2013) 546–551.
- [48] A.K. Jain, M.N. Murty, P.J. Flynn, Data clustering: A review, *ACM Comput. Surv.* 31 (3) (1999) 264–323.
- [49] G. Fung, A Comprehensive Overview of Basic Clustering Algorithms, Citeseer, 2001, pp. 1–37.
- [50] M. Papadakis, C. Henard, M. Harman, Y. Jia, Y. Traon, Threats to the validity of mutation-based test assessment, in: International Symposium on Software Testing and Analysis, 2016.
- [51] Y. Jia, M. Harman, Constructing subtle faults using higher order mutation testing, in: 2008 Eighth IEEE International Working Conference on Source Code Analysis and Manipulation, IEEE, 2008.
- [52] G. Fraser, A. Zeller, Mutation-driven generation of unit tests and oracles, *IEEE Trans. Softw. Eng.* 38 (2) (2012) 278–292.
- [53] C. Pacheco, M.D. Ernst, Randoop: Feedback-directed random testing for java, in: Companion to ACM SIGPLAN Conference on Object-Oriented Programming, ACM, 2007.
- [54] C. Pacheco, M.D. Ernst, Eclat: Automatic generation and classification of test inputs, in: Ecoop-Object-Oriented Programming, European Conference, Glasgow, UK, July, 2004.
- [55] M. Patrick, R. Alexander, M. Oriol, J.A. Clark, Subdomain-based test data generation, *J. Syst. Softw.* 103 (2015) 328–342.
- [56] D. Hao, L. Zhang, M.H. Liu, H. Li, J.S. Sun, Test-data generation guided by static defect detection, *J. Comput. Sci. Tech.* 24 (2) (2009) 284–293.
- [57] A.P. Mathur, W.E. Wong, An empirical comparison of data flow and mutation-based test adequacy criteria, *Softw. Test. Verif. Reliab.* 4 (1) (2010) 9–31.
- [58] W.E. Wong, A.P. Mathur, Reducing the cost of mutation testing: An empirical study, *J. Syst. Softw.* 31 (3) (1995) 185–196.
- [59] A.J. Offutt, G. Rothermel, C. Zapf, An experimental evaluation of selective mutation, in: Proceedings of 1993 15th International Conference on Software Engineering, IEEE, 1993.
- [60] A. Siami Namin, J.H. Andrews, D.J. Murdoch, Sufficient mutation operators for measuring test effectiveness, in: Proceedings of the 30th International Conference on Software Engineering, ACM, 2008.
- [61] A.M.R. Vincenzi, J.C. Maldonado, E.F. Barbosa, M.E. Delamaro, Unit and integration testing strategies for c programs using mutation, *Softw. Test. Verif. Reliab.* 11 (3) (2001) 249–268.
- [62] Y. Jia, M. Harman, Higher order mutation testing, *Inf. Softw. Technol.* 51 (10) (2009) 1379–1393.
- [63] E. Omar, S. Ghosh, D. Whitley, Subtle higher order mutants, *Inf. Softw. Technol.* 81 (2017) 3–18.
- [64] M. Polo, M. Piattini, I. García-Rodríguez, Decreasing the cost of mutation testing with second-order mutants, *Softw. Test. Verif. Reliab.* 19 (2) (2009) 111–131.
- [65] C. Sun, F. Xue, H. Liu, et al., A path-aware approach to mutant reduction in mutation testing, *Inf. Softw. Technol.* 81 (2017) 65–81.
- [66] X. Yao, D. Gong, Mutation testing based on comparison of paths, *Acta Electron. Sin.* 40 (1) (2012) 103–108.
- [67] K.N. King, A.J. Offutt, A fortran language system for mutation-based software testing, *Softw. - Pract. Exp.* 21 (7) (1991) 685–718.
- [68] R.H. Untch, A.J. Offutt, M.J. Harrold, Mutation analysis using mutant schemata, *ACM Sigsoft Softw. Eng. Notes* 18 (3) (1993) 139–148.
- [69] D. Gong, G. Zhang, X. Yao, F. Meng, Mutant reduction based on dominance relation for weak mutation testing, *Inf. Softw. Technol.* 81 (C) (2017) 82–96.
- [70] T.T. Chekam, M. Papadakis, T.F. Bissyand, Y.L. Traon, K. Sen, Selecting fault revealing mutants, *Empir. Softw. Eng.* 25 (1) (2020) 434–487.